

Saklı Markov Modelleri ile Kural Öğrenme Algoritmalarında Model Seçimi

Proje No: 107E127

Yrd.Doç.Dr. Olcay Taner YILDIZ
Özlem Aslan

ARALIK 2009
İSTANBUL

ÖNSÖZ

Proje iki temel kısımdan oluşmaktadır. Birinci kısımda kural öğrenme algoritmalarında model seçimi üzerinde çalışılmış, karar ağaçlarının VC-boyutu iki değişik yöntemle kestirilmiş ve bu kestirilen VC-boyut formülleri model seçiminde kullanılmıştır. İkinci kısımda Saklı Markov Modellerinde model seçimi problemi üzerinde çalışılmış, çapraz geçerleme tabanlı 5 farklı artırılmış model seçimi tekniği önerilmiş, yöntemlerin doğruluğu 4 farklı el işareti veri kümesi üzerinde gösterilmiştir. Bu çalışma Tübitak Elektrik Elektronik Enformatik Araştırma Grubu tarafından desteklenmiştir.

İÇİNDEKİLER

ÖNSÖZ	2
İÇİNDEKİLER.....	3
ŞEKİL LİSTESİ.....	4
TABLO LİSTESİ	5
ABSTRACT	6
ÖZET	7
1. GİRİŞ	8
1.1. VC-boyutu ve Karar Ağaçları.....	8
1.2. Saklı Markov Modelleri.....	10
2. YÖNTEM.....	12
2.1. Kapsamlı Arama Algoritması.....	12
2.2. Deneysel Dizayn Kullanarak VC-boyut Kestirimi	16
2.2.1. Düşüm Sınırlı Karar Ağacı Algoritması	16
2.2.2. Deneysel Dizayn	17
2.3. MSUMO: Yapı Seçimi İçin Meta Öğrenme Algoritması.....	19
2.3.1. Durum Uzayı Araması Olarak Yapı Öğrenme	19
2.3.2. MSUMO İşlemleri	21
2.3.3. Hatasızlığın ve Karmaşıklığın Kontrolü: MultiTest Algoritması	21
2.3.4. 5 MSUMO Varyantı	22
3. DENEYLER VE BULGULAR	24
3.1. Karar Ağaçlarında VC-Boyut Hesaplama	24
3.1.1. Kapsamlı Arama Sonuçları	24
3.1.2. Deney Dizaynı Sonuçları	28
3.1.3. VC-Boyut Kestirimleri Kullanarak Karmaşıklık Kontrolü	30
3.1.4. L-dallı Ağaçlar	34
3.2. Saklı Markov Modellerinde Model Seçimi.....	35
3.2.1. Deney Faktörleri ve Değerlendirme Kriterleri	35
3.2.2. Veri Kümeleri.....	37
3.2.3. Bulgular	39
4. TARTIŞMA VE SONUÇLAR.....	45
4.1. Karar Ağaçlarında VC-Boyut Hesaplama	45
4.2. Saklı Markov Modellerinde Model Seçimi.....	46
BİBLİYOGRAFYA.....	48

ŞEKİL LİSTESİ

Şekil 1. $d = 2$ ve $N = 3$ olan bir veri kümesi için tek değişkenli karar ağacının VC-boyutunun bulunması.....	13
Şekil 2. Tek değişkenli karar ağacının VC-boyutunu bulan kapsamlı arama algoritmasının sözde kodu: H : Karar ağacı hipotez sınıfı d : Veri kümesindeki boyut sayısı $data$: d boyutlu girdi için evrensel veri uzayı	14
Şekil 3. Düğüm sınırlı karar ağacı algoritmasının sözde kodu: M : Karar ağacındaki düğüm sayısı S : Veri kümesi.....	16
Şekil 4. Ortak dizayn algoritmasının sözde kodu: d : Veri kümesindeki girdi sayısı M : Karar ağacındaki düğüm sayısı D : Dizayn nokta sayısı.....	18
Şekil 5. 3 MSUMO varyantının durum uzayında örnek bir arama yapması. Mfw için, $HMM_{i,j}$ aşamasında dört aday model üretilir ($HMM_{i-1,j}$, $HMM_{i,j-1}$, $HMM_{i+1,j}$, $HMM_{i,j+1}$). $1-Fw$ 'da aday modeller basitten karmaşığa doğru işlenirken $1-Bw$ 'da karmaşıktan basite doğru işlenir.	23
Şekil 6. 2 MSUMO varyantının durum uzayında örnek bir arama yapması. Fw ve Bw için, $HMM_{i,j}$ aşamasında dört aday model üretilir ($HMM_{i-1,j}$, $HMM_{i,j-1}$, $HMM_{i+1,j}$, $HMM_{i,j+1}$). Fw 'da aday modeller basitten karmaşığa doğru işlenirken Bw 'da karmaşıktan basite doğru işlenir.	23
Şekil 7. 3 boyutlu veri kümeleri için oluşturulan karar ağaçlarının VC-boyutları. Sadece iç düğümler gösterilmiştir.	25
Şekil 8. 4 boyutlu veri kümeleri için oluşturulan karar ağaçlarının VC-boyutları. Sadece iç düğümler gösterilmiştir.	26
Şekil 9. İki karar ağacını birleştirerek yeni bir karar ağacı oluşturma işlemi. Şekildeki sayılar birleştirilmiş karar ağacına düğümlerin eklenme sırasını gösterir.	28
Şekil 10. Değişik özellik sayıları için ağaçtaki düğüm sayısına göre karar ağacının VC-boyutu.	29
Şekil 11. Değişik ağaç düğüm sayıları için veri kümesindeki özellik sayısına göre karar ağacının VC-boyutu.....	30

TABLO LİSTESİ

Tablo 1. 3 farklı fonksiyon için Noprune, CVprune, SRMexhaust ve SRMexper kullanılarak bulunan karar ağaçlarının hata oranları ile düğüm sayılarının ortalama ve standart sapmaları	32
Tablo 2. Veri kümesindeki farklı özellik sayıları için Noprune, CVprune, SRMexhaust ve SRMexper kullanılarak bulunan karar ağaçlarının hata oranları ile düğüm sayılarının ortalama ve standart sapmaları	32
Tablo 3. Farklı gürültü seviyeleri için Noprune, CVprune, SRMexhaust ve SRMexper kullanılarak bulunan karar ağaçlarının hata oranları ile düğüm sayılarının ortalama ve standart sapmaları.....	33
Tablo 4. Farklı veri kümesi büyüklükleri için Noprune, CVprune, SRMexhaust ve SRMexper kullanılarak bulunan karar ağaçlarının hata oranları ile düğüm sayılarının ortalama ve standart sapmaları.....	34
Tablo 5. Derecelerin hesaplanmasına bir örnek.	37
Tablo 6. Veri kümelerinin özellikleri.	37
Tablo 7. MSUMO varyantlarının istatistiksel test kullanmadan Derece(Sıra)'ları.	38
Tablo 8. MSUMO varyantlarının istatistiksel test kullanmadan Ortalama Hata(Durum Sayısı, Karışım Sayısı)'ları.	39
Tablo 9. MSUMO varyantlarının k-katlı t testi uygulanıp düzeltme uygulanmaksızın elde edilen Derece(Sıra)'ları.....	41
Tablo 10. MSUMO varyantlarının k-katlı t testi uygulanıp düzeltme uygulanmaksızın elde edilen Ortalama Hata (Durum Sayısı, Karışım Sayısı)'ları.	42
Tablo 11. MSUMO varyantlarının k-katlı t testi ile Holm düzeltmesi uygulanarak elde edilen Derece(Sıra)'ları.....	43
Tablo 12. MSUMO varyantlarının k-katlı t testi ile Holm düzeltmesi uygulanarak elde edilen Ortalama Hata (Durum Sayısı, Karışım Sayısı)'ları.....	44

ABSTRACT

In the first part of this project we propose two algorithms that calculate the VC-dimension of univariate decision trees with discrete features. The VC-dimension of the univariate decision tree with discrete features depends on (i) the VC-dimension values of its subtrees, (ii) the number of inputs, and (iii) the number of nodes in the tree. In the first algorithm, we form a training set of example trees whose VC-dimensions are calculated by exhaustive search. In the second algorithm, we estimate the VC-dimension of the univariate decision trees with binary features using optimized design. For both algorithms, we then fit a general regressor to estimate the VC-dimension of any univariate decision tree. To verify that these estimates are useful, we use them to get VC-generalization bounds for complexity control using SRM in decision trees, i.e., pruning. Our simulation results shows that SRM-pruning using the estimated VC-dimensions by either algorithm finds trees that are as accurate as those pruned using cross-validation. Comparing the two algorithms, the first algorithm using exhaustive enumeration is recursive and generalizes better than the second algorithm using experiment design on very large trees, whereas the latter is structure independent and faster than the former in terms of data generation.

In the second part of the project we define the problem of optimizing the architecture of a Hidden Markov Model (HMM) as a state space search and propose the MSUMO (Model Selection Using Multiple Operators) framework that incrementally modifies the structure and checks for improvement using cross-validation. There are five variants that use forward/backward search, single/multiple operators, and depth-first/breadth-first search. On four hand gesture data sets, we compare the performance of MSUMO with the optimal parameter set found by exhaustive search in terms of expected error and computational complexity.

Keywords: VC-dimension, model selection, decision trees, Hidden Markov Models, state space search

ÖZET

Bu projenin birinci bölümünde kesikli özellikli tek değişkenli karar ağaçlarının VC-boyutu'nu hesaplayan iki algoritma önerdik. Tek değişkenli karar ağaçlarının VC-boyutu (i) alt ağaçlarının VC-boyut değerlerine (ii) girdi sayısına (iii) karar ağacındaki düğüm sayısına bağlıdır. Birinci algoritmada kapsamlı arama tekniğiyle küçük karar ağaçlarının VC-boyutu hesaplanarak bir örneklem kümesi oluşturuldu. İkinci algoritmada optimize dizayn kullanılarak tek kesikli değişkenli karar ağaçlarının VC-boyutu kestirildi. Her iki algoritma için daha sonra genel bir kestirimci kullanılarak herhangi bir tek değişkenli karar ağacının VC-boyutunu hesaplayan formüller üretildi. VC-boyut kestirimlerinin faydalı olduğunu göstermek için, VC-boyut tahminleri karar ağaçlarında yapısal risk azaltma tekniği ile model seçiminde genelleştirme hatası bulmakta kullanıldı. Yaptığımız simülasyonların sonuçları her iki algoritma tarafından üretilen VC-boyut kestirimleri kullanılarak yapılan SRM-budama tekniği ile en az çapraz geçerleme kadar başarılı ağaçlar üretildiği saptadı. İki algoritmayı karşılaştırsak, kapsamlı arama tekniği kullanan birinci algoritmanın sonuçlarının özyinelemeli olduğunu ve optimize dizayn tekniğini kullanan ikinci algoritmaya göre büyük ağaçlarda daha iyi genellemeler yaptığını, buna karşılık ikinci algoritmanın ürettiği sonuçların ağacın yapısından bağımsız ve veri üretmede daha hızlı olduğunu görürüz.

Projenin ikinci bölümünde Saklı Markov Modellerinin yapısını en iyileme problemi durum uzayında arama yapmak olarak tanımlandı ve SMM'nin yapısını artırıcımlı değiştiren ve iyileşmeyi de çapraz geçerleme ile kontrol eden MSUMO altyapısı önerildi. MSUMO'nun beş değişik türü ileri/geri arama, tekli/çoklu işlemci ve derin/geniş arama tekniklerini kullanmaktadır. Dört el işareti veri kümesinde MSUMO'yu kapsamlı arama ile bulunan en iyi parametre kümesi ile model karmaşıklığı ve başarımlı açısından karşılaştırdık.

Anahtar Kelimeler: VC-boyutu, model seçimi, karar ağaçları, Saklı Markov Modelleri, durum uzayında aramak

1. GİRİŞ

1.1. VC-boyutu ve Karar Ağaçları

Yapay öğrenmenin temel amaçlarından biri verilen bir sınıflandırılmış eğitim kümesi üzerinden en iyi modeli seçmektir. Burada iyilik tanımı sınıflandırılmamış bilinmeyen test kümesi üzerindeki doğruluk ve sistemin karmaşıklığı olarak tanımlanır. Doğruluk ve karmaşıklık arasında bir bağlantı vardır: bir model çok karmaşıksa, eğitim kümesini kolaylıkla öğrenebiliriz ama ezberleme riski vardır, yani model genel bir kural öğrenmez, belli bir eğitim kümesini öğrenir ve bilinmeyen bir test kümesinde çok fazla hata verir. Eğer model çok basitse, eğitim kümesi bile öğrenilemeyebilir ve hata eğitim ve test kümesinde çok fazla olabilir.

Vapnik-Chervonenkis (VC) teorisi (VAPNIK, 1995) bu model seçimi problemini kestirimci (predictive) öğrenme olarak tanımlar ve bunu çözmek için matematiksel bir yapı kullanılmasına olanak sağlar. VC-teorisi ve Yapısal Risk Azaltma (SRM) yöntemine göre, veri kümesinden kestirilmiş VC genelleştirme sınırı (VC generalization bounds) en aza indirgenerek en iyi model bulunabilir. Bir modelin VC genel aralıklarını hesaplamak için, (i) modelin karmaşıklığını gösteren VC-boyutuna, (ii) modelin eğitim kümesi üzerindeki hatasına ihtiyacımız var. Örneğin, iki sınıf problemi için, modelin genelleştirme aralığı şu şekilde verilebilir (CHERKASSKY VE MULIER, 1998)

$$E_g = E_t + \frac{\varepsilon}{2} \left(1 + \sqrt{1 + \frac{4E_t}{\varepsilon}} \right)$$

E_t öğrenme hatasıdır ve ε

$$\varepsilon = a_1 \frac{V \left[\log \left(a_2 \frac{N}{V} \right) + 1 \right] - \log v}{N}$$

şeklinde ifade edilir. Formüllerde, V modelin VC-boyutunu, v ise güvenilirlik seviyesini (confidence level) ifade eder. a_1 ile a_2 deneysel olarak bulunmuş sabit değerlerdir.

Bir veri kümesi verildiğinde, N veri noktası 2^N farklı yolla pozitif ve negatif olarak sınıflandırılabilir. Bu sınıflandırmalardan (labelings) herhangi bir tanesi için, pozitif ve negatif veri noktalarını H hipotez sınıfından bir h hipotezini kullanarak ayırabiliyorsak, H 'nin N veri noktasını ayırdığını (shatters) söyleyebiliriz. H tarafından

ayrılabilir veri noktası sayısının en büyük değerine H hipotez sınıfının VC-boyutu denir ve aynı zamanda VC-boyutu H hipotez sınıfının kapasitesidir de denir.

Bu çalışmada, biz hipotez sınıfı olarak karar ağaçlarını kullanıyoruz. Karar ağaçları ağaç-tabanlı yapılar olup (i) her düğüm bir kesme fonksiyonu ($f_m(x)$) içerir (ii) yaprak olmayan düğümlerin her dalı (çocuğu) kesme fonksiyonunun olası bir değerine karşılık gelir (iii) her yaprak bir sınıfı gösterir. $f_m(x)$ kesme fonksiyonu x veri vektörünün tek bir boyutunu kullanıyorsa, karar ağacı tek değişkenli karar ağacıdır (QUINLAN, 1993). ID3 en çok bilinen tek kesikli değişkenli karar ağacı algoritmalarından biridir (QUINLAN, 1986).

Bir karar ağacının en iyi karmaşıklığını bulmak önemlidir. Karmaşık ağaçlarda, genel bir kural öğrenmek yerine eğitim kümesinin belli bir kısmını öğreniriz ve bu da bilinmeyen test veri kümesinde çok hata verir. Çok basit ağaçlarda, eğitim kümesi bile öğrenilemeyebilir ve hata eğitim kümesinde ve test kümesinde yüksek olabilir.

Çok tanınan bazı sınıflandırıcılarının VC-boyutu bilindiği halde, bütün sınıflandırıcıların VC-boyutunu elde etmek mümkün değildir (VAPNIK, 1995). Geriye kalan tek çözüm deneysel olarak bu sınıflandırıcıların VC-boyutunu kestirmektir (VAPNIK VE DİĞERLERİ, 1994), (SHAO VE DİĞERLERİ, 2000). Kestirmede kullanılan yaklaşım, iki sınıflı sınıflandırma problemlerinde sınıflandırıcının eğitim hatasının 0,5 (iki sınıflı bir problemde sınıflandırıcının yapabileceği en büyük hata oranı)'ten farkının o sınıflandırıcının VC-boyutu ve veri kümesindeki örnek sayısı ile ilişkili olduğu gerçeğine dayanır. VAPNIK VE DİĞERLERİ, 1994 bu ilişki için teorik bir formül üretmişlerdir. Yapay olarak üretilen veri kümeleri üzerinde yapılan deneylerdeki hata frekansı kullanıp teorik formülün katsayılarını elde etmiş ve bu yöntem ile sınıflandırıcıların VC-boyutunu kestirmenin yolunu açmışlardır.

SHAO VE DİĞERLERİ, 2000 ise bu çalışmanın üzerine optimize edilmiş deney dizaynını kullanarak elde edilebilecek VC-kestirimlerini iyileştirmeyi başarmışlardır. SHAO VE DİĞERLERİ, 2000'nin algoritması (VAPNIK VE DİĞERLERİ, 1994)'de tanımlanmış ortak deney dizaynı ile başlayıp dizayn noktaları üzerinde ikili değiştirme yapar ve optimize edilmiş deney dizaynını bulurlar. SHAO VE DİĞERLERİ, 2000 çalışmalarında iyi ve kötü dizayn noktalarını bulmak için ortalama hatanın karesi (mean square error, MSE) kullanmışlardır.

VC-boyutunu kestirdikten sonra, kestirdiğimiz boyutu VC genelleme aralığının içine koyabilir ve Yapısal Risk Azaltma tekniği ile model seçimi yapılabilir. SHAO VE

DİĞERLERİ, 2000 ilk olarak VC-boyutunu cezalandırılmış (penalized) doğrusal kestirimciler için deneysel olarak kestirip regresyonda model seçimi için kullandılar. Deneysel sonuçlarına göre, optimize edilmiş deney dizaynıyla bulunmuş VC-boyutu iki klasik model seçimi kriteri, Akaike'nin tahmin hatası ve genelleştirilmiş çapraz geçiremeye göre, daha iyi model seçimi performansına sahiptir (CHERKASSKY VE MA, 2003).

Bir karar ağacının VC-boyutu ile ilgili genel bir kural yoktur, ama yukarıdaki yöntemle benzer bir yaklaşımla onu kestirebiliriz. Bu çalışmada en kolay durum olan tek değişkenli kesikli özellikli (discrete features) karar ağaçlarına odaklandık. İnanıyoruz ki tek değişkenli ve kesikli özellikli karar ağaçlarının VC boyutu veri kümesinin boyuna, karar düğümü sayısına ve ağaç yapısına bağlıdır. İlk yaklaşımımız şudur: Bir çok basit ağaçlar için, kapsamlı arama algoritması (exhaustive search algorithm) kullanarak gerçek VC-boyutunu hesaplıyoruz. İkinci yaklaşımımızda, tek değişkenli karar ağaçlarının VC-boyutunu optimize edilmiş deney dizaynı ve ortak deney dizaynını kullanarak kestiriyoruz. Sonra bunları eğitim kümesi olarak kullanarak, herhangi bir ağacın VC-boyutunu bulabileceğimiz bir regressor oturtuyoruz.

Bu VC-boyutu kestirimlerinin gerçekten iyi kestirimler olup olmadıklarının sağlamasını yapmak için bu kestirimleri karar ağaçlarını budamada kullanıyoruz. Bizim burada üstünde durduğumuz nokta ortak ağaç yapısını paylaşan ve sadece iç düğümlerde test edilen değişkenlerin farklı olduğu karar ağaç ailesi hipotez sınıfının VC-boyutudur.

1.2. Saklı Markov Modelleri

Her sınıflandırma algoritması veri kümesi hakkında bir varsayıma sahiptir. Ayrıca her algoritmanın farklı yapısı ve en iyi performansı bulmak için ayarlanan parametreleri vardır. Genelde optimum performans için öğrenilen parametre sayısı çok olduğu için, farklı parametreleri ve opsiyonları denemek ve optimize etmek için hesaplama açısından sistematik ve uygulanabilir bir yola ihtiyacımız vardır.

Saklı Markov Modellerinde, grafik model, saklı durum sayısı, saklı durumlardaki karışım eleman bileşen (mixture component) sayısı (eğer durum karışım modeline sahipse) ve saklı durumlar arasındaki bağlantısallık ile tanımlanır. Model yapısı bir kere sabitlendiğinde, Baum Welch algoritması (RABINER, 1989)

ağıdaki durumlar arası geçiş, örnek listesi ve ilk durum olasılıkları öğrenilebilir. Saklı Markov Modelleri hakkındaki bir giriş materyali (RABINER, 1989) ve (GHAHRAMANI, 2001)'de bulunabilir. Bu çalışmada, her düğümde normal karışım ve soldan-sağa model (lr), soldan-sağa-döngü (lrl) ve tam model (full) diye adlandırılan 3 değişik yapıda SMM kullanıyoruz. Geçmiş çalışmalarda (ARAN VE DİĞERLERİ, 2009), MOST algoritmasını önerdik ve çok katmanlı sinir ağındaki uygulamasını gösterdik.

SMM'nin en iyi modeli altta yatan fonksiyonu öğrenebilecek kadar büyük ve genelleme yapabilecek kadar küçük bir grafik modeldir. Daha küçük bir model problemi iyi öğrenemeyebilir, diğer taraftan en iyi modelden daha büyük bir model de ezberler, iyi genelleyemez ve daha karmaşık bir yapıya sahiptir. Buradaki önemli nokta sapma (bias) ve varyans arasındaki ilişkidir; birinden kazanırken diğerinden kaybederiz. (HASTIE VE DİĞERLERİ, 2001): küçük ve / veya kolay bir model öğrenemez ve veriyi öğrenmede başarısız olur (sapma yüksek ve varyans düşük). Büyük ve/veya karmaşık bir model ezberler; veriyi öğrenir (sapma düşük) ama gürültüyü de öğrenir (varyans yüksek). En iyi yapı düşük sapma ve düşük varyanslı olmalıdır, öyle ki model eklenen gürültüyü değil verinin altında yatan fonksiyonu öğrenir.

Veri madenciliği uygulamaları ve endüstride kullanılan yapay öğrenme araçları sayısının artması çoğu uzman olmayan ve algoritmalar ile modeller hakkında bilgiden yoksun kullanıcıların sayısını arttırdı. Bu kişiler problemi ve eldeki veriyi verip yapay öğrenme araçları tarafından üretilen kabul edilebilir ve anlaşılır bir çözüm beklerler. Özellikle SMM durumu için, düğüm sayısını ve / veya her düğümdeki fonksiyonu uzman bir kullanıcı için bile kolayca kestiremez. Bu nedenle, kullanıcıya kendini göstermeden modeli en iyileyen bir metodolojiye ihtiyaç var; bu metodoloji en iyi modeli bulamayabilir ama kabul edilebilir bir zamanda yeterli düzeyde iyi bir yapı döndürür. SMM'ler biyoinformatikte (ikinci yapı kestirimcileri olarak (MARTIN VE DİĞERLERİ, 2005)) ve konuşma işlemede (ses modelleme araçları olarak (REYES VE DİĞERLERİ, 2003)) uzun zamandır kullanılmaktadır.

BIEM VE DİĞERLERİ, 2002 SMM eniyilemesi için Bayes tabanlı model seçimi kriteri önerdiler. Genel Bayes Bilgi Kriteri'nde (BIC), SMM modelinin her bir parametresinin ön olasılıkları için çok değişkenli normal dağılımı kullanılır. Diğer taraftan, BIEM VE DİĞERLERİ, 2002 her bir parametreyi farklı bir dağılımla modellediler. Önerdikleri algoritmayı el yazısı karakterlerinin tanınması problemi

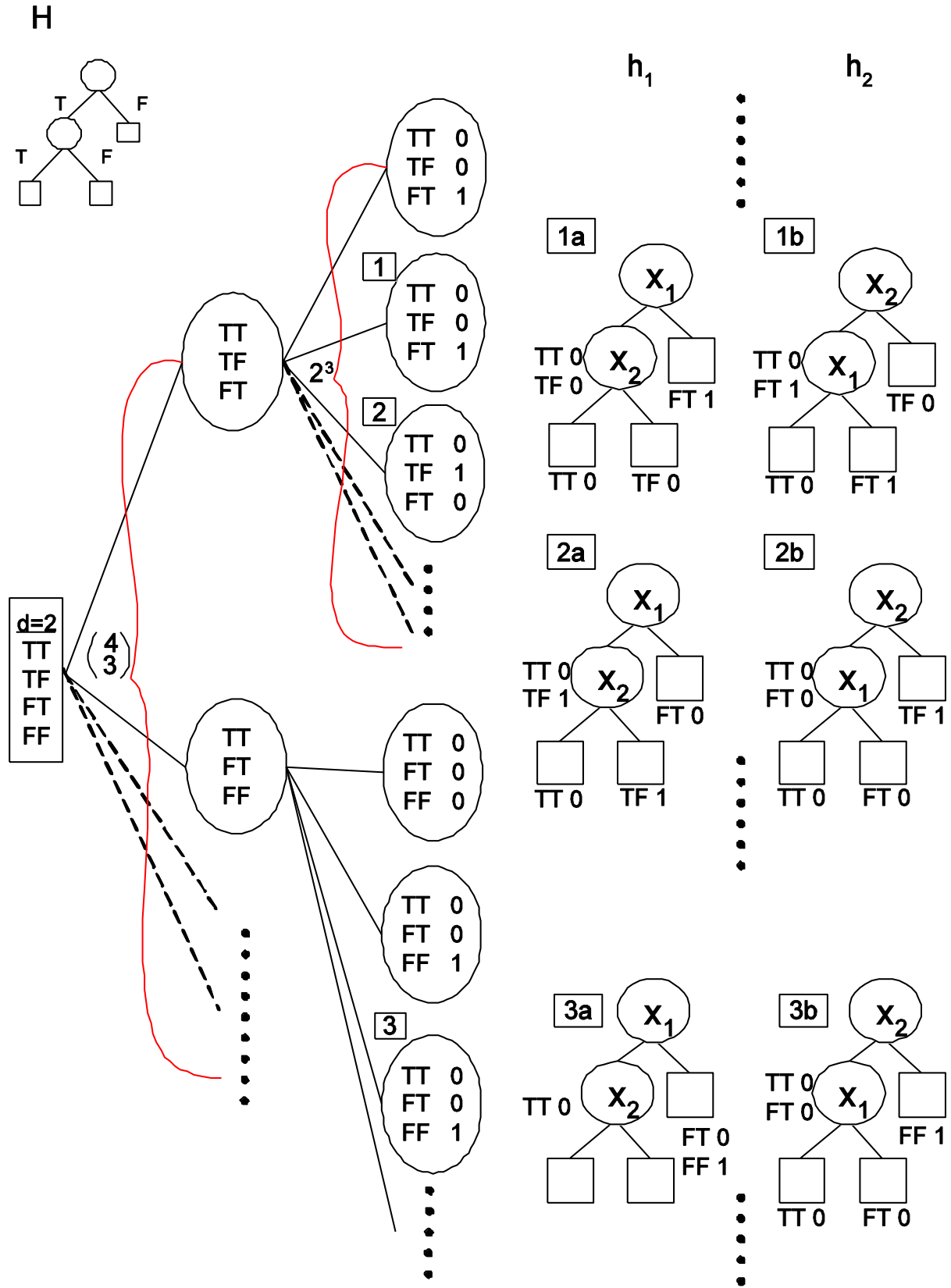
üzerinde BIC ile karşılaştırdılar. Aynı yazarın diğer bir çalışmasında (BIEM, 2003), ayrımcı (discriminative) bilgi kriteri (DIC) SMM model seçimi için önerildi. DIC her bir sınıf için model log olabilirliği ve diğer sınıfların tüm modellerinin ortalama log olabilirliği arasındaki farkı en büyük yapan SMM modelini seçer. Başka bir kriter de yukarıdan-aşağıya ön olasılık / arka olasılık (prior / posterior) yayılım algoritması ile en iyi ağaç yapılı SMM'i seçmek için kullanılan kestirimci (predictive) bilgi kriteri'dir (PIC) (CHIEN VE FURUI, 2005).

2. YÖNTEM

2.1. Kapsamlı Arama Algoritması

H hipotez sınıfından bir karar ağacının VC-boyutunu bulmaya çalıştığımızı düşünelim. d boyutlu ve her boyutu iki değerli bir veri kümesi için, olası tüm farklı veri noktaları sayısı 2^d dir. Kapsamlı arama algoritmasının temel fikri şu şekilde açıklanabilir: Bu 2^d tane veri noktasının N elemanlı bir alt kümesini aldığımızda, eğer bu N tane noktanın olası işaretlemelerinden her biri için bizim önceden varsaydığımız onu doğru sınıflandıran H hipotez sınıfına ait bir tane h hipotezi varsa, karar ağacının VC-boyutunun en az N olduğunu söyleyebiliriz. H nin VC-boyutu olabilecek en büyük N 'dir.

Bir veri kümesi için $d = 2$ tane özellik ve $N = 3$ data noktası için kapsamlı arama algoritmasının örnek bir gösterimi Şekil 1'de verilmiştir. 2 düğümlü H hipotez sınıfını tanımlayan bir karar ağacı yapısının VC-boyutunu bulmak istiyoruz. Bu nedenle, H hipotez sınıfına ait sadece iki benzer olmayan (nonisomorphic) h_1 ve h_2 hipotez sınıfımız var. Birinci hipotez sınıfı h_1 için, birinci özellik birinci düğümdeki ayırmada kullanılır ve ikinci özellik ikinci düğümdeki ayırmada kullanılır. İkinci hipotez sınıfı h_2 için, ikinci özellik birinci düğümdeki ayırmada kullanılır ve birinci özellik ikinci düğümdeki ayırmada kullanılır.



Şekil 1. $d = 2$ ve $N = 3$ olan bir veri kümesi için tek değişkenli karar ağacının VC-boyutunun bulunması

Girilen özellik sayısı 2'dir. Bu nedenle farklı veri noktaları sayısı $2^2 = 4$ 'tür. Biz örnekte $N = 3$ veri noktasını taradığımız için, 3 noktanın 4 noktadan gelen tüm kombinasyonları Şekil 1'de gösterilmiştir. Her bir $N = 3$ 'lük data kombinasyonunun her olası işaretlemesi için h_1 veya h_2 hipotezlerinden birinin doğru sınıflandırıp sınıflandırmadığını kontrol ediyoruz.

- İlk data kombinasyonuna denk gelen ilk işaretleme için, (Durum 1'de gösterildi), h_1 ve h_2 hipotezlerinin ikisi de veriyi doğru sınıflandırır. (Ağaçlar 1a ve 1b). Sadece bir hipotezin yeterli olduğunu dikkate almalıyız.
- Birinci data kombinasyonuna denk gelen ikinci işaretleme için (Durum 2'de gösterildi), h_1 ve h_2 hipotezlerinin ikisi de veriyi doğru sınıflandırır. (Ağaçlar 2a ve 2b). Ağaç 2b'deki ikinci düğüm veriyi sınıflandırmak için gerekmiyor.
- Üçüncü veri kombinasyonuna denk gelen birinci işaretleme için (Durum 3'te gösterilmiş), h_1 hipotezi veriyi doğru sınıflandıramıyor (Ağaç 3a), ama ikinci hipotez h_2 veriyi doğru sınıflandırıyor (Ağaç 3b).

```

VCBoyut KapsamlıArama (H, d, data)
1.N = 1
2.while TRUE
3.    dataComb = getDataCombination(data, N)
4.    successful = FALSE
5.    while dataComb != NULL
6.        classifiedAllCombinations = TRUE
7.        classComb = getClassCombination(N)
8.        while classComb != NULL
9.            if not treeClassify($H$, dataComb, classComb)
10.                classifiedAllCombinations = FALSE
11.                break
12.            classComb = getNextClassCombination(N)
13.        if classifiedAllCombinations
14.            successful = TRUE
15.            break
16.        else
17.            dataComb = getNextDataCombination(data, N)
18.    if successful
19.        N = N + 1
20.    else
21.        break
22.return N - 1

```

Şekil 2. Tek değişkenli karar ağacının VC-boyutunu bulan kapsamlı arama algoritmasının sözde kodu: H : Karar ağacı hipotez sınıfı d : Veri kümesindeki boyut sayısı $data$: d boyutlu girdi için evrensel veri uzayı

Tek değişkenli ve ikili özellikli karar ağacının VC-boyutunu bulmak için kullanılan sözde kod Şekil 2'de verilmiştir. Verilen d boyutlu bir veri kümesi için, ilk

önce tüm olası kombinasyonları yaratmak yerine, N veri noktası içeren olası tüm kombinasyonları yaratıyoruz (Satır 3 ve 17). Her bir veri kombinasyonu için, her bir olası işaretlemeyi oluşturuyoruz (Satır 7 ve 12). Veri kombinasyonunun her bir olası işaretlemesi için, veriyi doğru sınıflandıran H hipotez sınıfından gelen bir h hipotezi olup olmadığını kontrol ediyoruz (Satır 9). Hipotez kontrolü karar ağacının düğümlerinde kullanılan özellikleri değiştirerek yapılmıştır. Böyle bir hipotez yoksa (Satır 10 ve 11), sınıflandırmayı durdururuz ve bir sonraki veri kombinasyonu ile aramaya devam ederiz (Satır 17). Böyle bir veri kombinasyonu varsa, bir sonraki işaretlemeye geçeriz (Satır 12). Eğer bir veri kombinasyonunun olası tüm işaretlemeleri için bir h karar ağacı bulabiliyorsak (Satır 13 ve 14), N 'yi bir arttırırız (Satır 18 ve 19) ve aramaya devam ederiz. Eğer 2^d 'nin tüm N tane alt kümeleri aranır ve hiçbir alt küme tüm işaretlemeler için sınıflarına ayrılamazsa, arama bitmiştir, VC-boyut $N - 1$ olarak alınır.

Hesaplama arttırımsaldır, eğer bir veri kombinasyonu bir veri büyüklüğü için sınıflarına doğru ayrılmışsa, veri büyüklüğü bir arttırılır ve algoritma aramaya devam eder. Bir veri büyüklüğü N ve boyutu d için, $\binom{2^d}{N}$ tane olası veri kombinasyonu vardır. Her bir N büyüklüğündeki veri kombinasyonu için, 2^N tane olası işaretleme vardır. En kötü durumda, karar ağacı hipotez sınıfı H 'nin her bir elemanını kontrol etmemiz gerekir, algoritmanın hesaplama karmaşıklığı

$$\sum_{N=1}^V \binom{2^d}{N} 2^N |H|$$

olur.

Bu nedenle kapsamlı arama algoritmasını sadece birkaç H ve küçük d 'li ve $|H|$ 'li durumlar için çalıştırıyoruz. Örneğin, derinliği 4 ve 4 özellikli karar ağacını 8 çekirdekli bir bilgisayarda bitirmek 2 gün alır, buna karşın derinliği 5 ve 5 özellikli karar ağacını taramak 10^{13} gün alır. Daha sonra elde ettiğimiz bu karar ağacı veri kümesine bir regressor oturtuyoruz ve diğer karar ağaçları için geçerli olup olamayacağını kontrol ediyoruz.

2.2. Deneyisel Dizayn Kullanarak VC-boyut Kestirimi

2.2.1. D ğ m Sınırlı Karar Ađacı Algoritması

M d ğ ml  bir karar ađacının VC-boyutunu kestirmek i in, ilk  nce veri k meleri S 'den gelen M d ğ ml  karar ađacını yaratan bir algoritmaya ihtiyacımız var. Genel olarak, karar ađacı algoritmalarında  ğrenme hatası sıfır olana kadar karar ađacı oluřturulmaz, erken sonlamalarla karar ađacı k   k tutulur ( n budama) veya karar ađacı  ğrenme hatası sıfır olana kadar  ğrenilir daha sonra budanır (arka budama).

Arka budamada, ge erleme k mesinde, alt ađacın performansını alt ađacın yerine yaprak yerleřtirildiđinde elde edilen performansla karřılařtırırız. Ezberleme varsa, daha karmařık ađacın g r lt y   ğrenmesini bekleriz ve basit yaprak modelinden daha k t   alıřmasını bekleriz. Burada ge erleme k mesi budamadan sonraki d ğ m sayısını belirler ve d ğ m sayısını  nceden belirlemek m mk n deđildir.

```
1. KararAđacı D ğ mSınırlıKararAđacı(M; S)
2.   MinHeap = {}
3.   NodeExpanded = 0
4.   L = Loglikelihood(S)
5.   AddToHeap(MinHeap, (RootNode, S))
6.   while NodeExpanded < M
7.     (Node, Data) = RemoveFromHeap(MinHeap)
8.     NodeExpanded = NodeExpanded + 1
9.      $f_m(x) = \text{FindBestSplit}(\text{Data})$ 
10.    (Left, Right) = Expand(Node, Data,  $f_m(x)$ )
11.     $L_L = \text{Loglikelihood}(S_L)$ 
12.    AddNodeToMinHeap(MinHeap, (Left,  $S_L$ ))
13.     $L_R = \text{Loglikelihood}(S_R)$ 
14.    AddNodeToMinHeap(MinHeap, (Right,  $S_R$ ))
15.   return RootNode
```

řekil 3. D ğ m sınırlı karar ađacı algoritmasının s zde kodu: M : Karar ađacındaki d ğ m sayısı
 S : Veri k mesi

D ğ m sınırlı ađac algoritması řekil 3'te g sterilmiřtir. Her bir ařamada, M d ğ m m z olana kadar logbenzerliđi en az olan d ğ m  a ıyoruz. A ılmamıř d ğ mler min-heap'te tutulur. N toplam  rnek sayısı, n_i i . sınıfın  rnek sayısı, n^L ve n^R sırasıyla sol ve sađ dalları se en  rnek sayısı olduđunda, logbenzerlik řu řekilde ifade edilir:

$$\mathcal{L} = \sum_{i=1}^2 n_i^L \log \frac{n_i^L}{n^L} + \sum_{i=1}^2 n_i^R \log \frac{n_i^R}{n^R}$$

Algoritma heap'i sıfırlayarak başlar (Satır 2). Sonra tüm veri kümesi kök düğüm heap'ine koyulur (Satır 4 ve 5). Açılan düğüm sayısı M 'den küçükse (Satır 6), en az logbenzerlikli düğümü heap'ten çıkarırız (Satır 7), en iyi karar fonksiyonunu buluruz (Satır 9), düğümü sol ve sağ çocuğunu yaratmak üzere açarız (Satır 10) ve bu iki yeni düğümü heape ekleriz (Satır 11-14).

2.2.2. Deneysel Dizayn

Bu bölümde, VC-boyutunu kestirmede kullandığımız iki deney dizayn metodolojisini (ortak ve optimize edilmiş deney dizaynları) gözden geçiriyoruz.

VC teoriye göre, iki bağımsız sınıflandırılmış veri kümesinden gelen hatanın en büyük sapması, $\varepsilon(n)$, $\varphi(n / V)$ ile sınırlandırılmıştır. Burada $\rho \geq 0.5$ ve $\rho = n / V$ iken

$$\varphi(\rho) = a \frac{\ln(2\rho) + 1}{\rho - k} \left(\sqrt{1 + \frac{b(\rho - k)}{\ln(2\rho) + 1}} + 1 \right)$$

'dir ve $a = 0.16$, $b = 1.2$, $k = 0.14928$ değerleri deneysel olarak elde edilmiştir (VAPNIK VE DİĞERLERİ, 1994).

VAPNIK VE DİĞERLERİ, 1994'e göre, bu sınır dardır, böylelikle $\varepsilon(n)$ 'nin $\varphi(n / V)$ 'e yaklaşık olarak eşit olduğunu düşünebiliriz.

Deney dizaynı dizayn noktaları ve her bir nokta için yapılan deneylerden oluşur. Bizim durumumuzda, bir deney n nokta için bir epsilon kestirimi bulmak için kullanılır. Eğer dizayn noktalarındaki deney sayıları eşitse, bu ortak dizayn diye adlandırılır. Optimize edilmiş dizaynda, amaç her dizayn noktasındaki optimum deney sayısını bulmaktır (LI VE WU, 1997).

VC-boyutunu kestiren ortak dizaynın sözde kodu Şekil 4'te verilmiştir. Tek bir deneyde, hatanın en büyük sapmasını bulmak için, karar ağacı tüm S kümesinde eğitilir (Satır 8), ve verinin ilk S_1 parçasındaki hata en aza indirgenir (Satır 10) ve ikinci S_2 parçasındaki hatayı en fazla yapmak için sınıflar tersine çevrilerek test edilir (Satır 9, 11).

```

1. VcEstimate OrtakDizayn(d; M; D)
2.   for i = 1 to D
3.       ni = Sample size where 0.5 ≤ ni / V ≤ 30
4.       for j = 1 to mi
5.           Generate a random sample S of size 2ni
6.           Split S into two samples of equal size (S1, S2)
7.           Flip class labels of S2
8.           DT = D ğ mSınırlıKararAğacı (M, S)
9.           Flip class labels of S2 back again
10.          E(S1) = Error rate of DT on S1
11.          E(S2) = Error rate of DT on S2
12.          ε(ni,j) = |E(S1) - E(S2)|
13.          ε(ni) = Σj=1mi ε(ni,j) / mi
14.      V = Best fit between φ(ni / V) and ε(ni)
15.  return V

```

Şekil 4. Ortak dizayn algoritmasının sözde kodu: d : Veri kümesindeki girdi sayısı M : Karar ağacındaki düğüm sayısı D : Dizayn nokta sayısı

Ortak dizaynda, her bir i veri noktasında, m_i deney yapılır (Satır 4) ve epsilon kestirimlerinin ortalaması hesaplanır (Satır 13). Kestirimlerin değişkenliğini azaltmak için, her noktadaki veri sayısı, n_i , farklıdır, ve

$$0.5 \leq \frac{n_i}{V} \leq 30$$

aralığında seçilir. M düğümlü karar ağacının VC-boyutu V 'dir (Satır 14) ve şu fonksiyonu en aza indirger

$$\sum_{i=1}^D \left[\bar{\epsilon}(n_i) - \Phi\left(\frac{n_i}{V}\right) \right]^2$$

Aksine, optimize edilmiş dizaynda, ilk fazda, ilk VC-boyut kestirimlerini her bir dizayn noktasında eşit deney sayılı ortak dizayn kullanarak elde ederiz. Oturtulmuş eğrinin ortalama hatasının karesi şu andaki VC-boyutu V 'yi kullanarak bulunur. (Denklem 6'da verildiği gibi) Dizaynı optimize etmek için, hangi dizayn noktasının iyi hangisinin kötü olduğunu bulmamız gerekir. Bir dizayn noktasının iyiliği MSE'ye olan katkısından hesaplanır. İyi dizayn noktası MSE'de azalmaya yol açar, kötü dizayn noktaları artmaya yol açar. Her bir noktanın katkısını hesapladıktan sonra, katkılar azalacak biçimde sıralanır, en iyi nokta ilktir ve en kötü nokta sonuncudur.

Optimize edilmiş dizaynın temel aşamasında, en kötü ve en iyi dizayn noktaları arasındaki değişimler yapılır. En kötü dizayn noktasından bir deney çıkarırız ve onu en iyi dizayn noktasına ekleriz ve yeni VC- boyutu V' ve MSE' yi hesaplarız. Eğer değişim daha az bir MSE'ye neden oluyorsa, değişim başarılıdır ve yeni V' ile

ana döngüye devam edebiliriz. Eğer değişim daha fazla bir MSE'ye neden oluyorsa, değişim başarısızdır ve değişimi tersine çevirmeliyiz. Bu durumda, daha az bir MSE bulana kadar, en kötü ve ikinci (üçüncü, dördüncü, vs.) en iyi dizaynlar arasındaki ikili değişimleri aramaya devam etmeliyiz.

Bir dizayn noktasındaki deneylerin sayısının toplam deney sayısının yüzde 25'inden fazla olamayacağına dikkat etmek gerekir. Bir dizayn noktası bu limite ulaşırsa, doyumluğa ulaşır ve ikili değişim yapamaz. Bir limit yoksa, her zaman tüm deneyleri içeren olası bir design noktası vardır ve VC-boyut kestirimi yüksek oranda hatalı olur. Optimize edilmiş dizaynın ana döngüsü, yani, bulma ve sıralama katkıları, ikili değişimler yapma, her dizayn noktası negatif bir katkıya sahip olursa veya pozitif katkıya sahip olan tüm noktalar doyumluğa ulaşmışsa biter.

2.3. MSUMO: Yapı Seçimi İçin Meta Öğrenme Algoritması

2.3.1. Durum Uzayı Araması Olarak Yapı Öğrenme

SMM'nin yapısını optimize etmek için SMM'nin olası tüm yapılarının oluşturduğu durum uzayını aramak gerekir. Bu çalışmada, model yapısı sabitlendiğinde (soldan-sağa model(lr), soldan-sağa-döngü (lr) ve tam model), arama uzayı düğüm sayısının tüm olası kombinasyonlarını ve düğümlerdeki tüm olası karışım sayılarını içerir. Optimize edilen parametre sayısı artarsa, en iyisini seçmek için tüm olası yapıları denemek ve değerlendirmek mümkün olmaz, bu nedenle en yakın optimum çözümü bulmak için arama uzayında en az olası durumu ziyaret eden bulgusal bir stratejiye ihtiyaç vardır. Parametrelerin sürekli olduğu parametre ayarlamalarında, farklı yapı sayısı sonsuzdur.

SMM yapısını değiştiren işlemleri tanımlıyoruz ve bir durumdan diğerine geçmesine izin veriyoruz. Örneğin, ADD-1 operatörü bir düğüm daha ekler ve bu işlemi uygulamak bizi $SMM_{4,1}$ durumundan $SMM_{5,1}$ durumuna götürür, yani 4 düğümlü bir SMM'den 5 düğümlü bir SMM'e geçmek. Bu nedenle, arttırımlı algoritmaları basit bir ilk durumdan başlayan ve ADD işlemlerini kullanan *ileri aramayı* (forwardsearch) gerçekleştiren algoritmalar olarak düşünüyoruz. Diğer taraftan, azaltımlı/budama algoritmaları karmaşık bir durumdan başlayarak ve durum(lar)/karışım(lar)ı budamak için REMOVE işlemlerini kullanarak *geri aramayı* gerçekler. *Değişen arama (Floating search)* ADD ve REMOVE işlemlerinin ikisini de kullanmayı sağlar.

Bir işlem uyguladıktan sonra, durum değerlendirme fonksiyonu seçim kriterine göre bir sonraki durum ile şimdiki durumun performansını karşılaştırır ve işlemi performans ölçütünün gelişip gelişmemesine göre kabul eder veya reddeder. Bu durum değerlendirme fonksiyonu bir sonraki duruma ait SMM'i öğrenir, gerçeller ve hem genelleme hatasını hem de karmaşıklık ölçütünü hesaba katarak bir model seçim kriteri kullanır, böylelikle daha doğru ve kolay yapılar tercih edilir.

Bir önceki çalışmamızda olduğu gibi (ARAN VE DİĞERLERİ, 2009), optimum yapıyı bulma problemini durum uzayında bir arama olarak tanımladığımızda, elimizde 5 değişik seçenek olacaktır:

- 1) İlk durum. İlk durum olarak tek bir normal dağılım ve tek saklı durum içeren bir SMM seçersek, algoritma temel olarak arttırımlıdır ve yapıya saklı durumlar ve/veya karışımlar ekler. N saklı durumlu bir SMM'den başlarsak ($SMM_{N,1}$), algoritma azaltımlı olur ve durum sayısı ile karışımındaki normal dağılımları azaltır.
- 2) Durum geçiş işlemleri. İşlemler durum/karışım ekler veya çıkarır. Daha hızlı bir yakınsama için, durum uzayında çoklu ünite ekleyerek/çıkararak (mesela ADD-5 5 saklı durum ekler, ADD-1 tek durum ekler) daha uzun atlamalar yapan operatörler kullanabiliriz. Bu çalışmada çoklu atlamalar kullanmıyoruz.
- 3) Çoklu arama. Tek aday yapısı veren tek bir işlemimiz olabilir veya çoklu adaylar elde etmek için çoklu işlemler uygulayabiliriz. Çoklu işlem durumunda, genişlik-öncelikli, derinlik-öncelikli, en iyi-öncelikli arama veya diğer türevleri kullanılabiliriz.
- 4) Durum değerlendirme fonksiyonu. Çapraz geçерleme ve ilgili istatistiksel test kullanılabilir veya bilgi teorisi ölçütlerine dayalı model seçimi kriterlerini kullanabilir, mesela Akaike'nin bilgi kriteri (AIC), minimum tanım uzunluğu (MDL), Bayesçi bilgi kriteri (BIC) veya yapısal risk azaltma(SRM) (HASTIE VE DİĞERLERİ, 2001).
- 5) Durma şartı. Açık olan şart hiç bir aday şu andaki en iyiyi iyileştirmezse aramayı durdurmaktır. Diğer bir olası şartsa hata belli bir değerin altına inerse veya belli aşama sayısına ulaşıldığında durmaktır.

2.3.2. MSUMO İşlemleri

MSUMO şu işlemleri destekler (model karmaşıklığı üzerindeki etkileri artarak sıralanmış):

- 1) REMOVE-1: SMM'den tek bir saklı durum çıkarır
- 2) ADD-1: SMM'e tek bir saklı durum ekler
- 3) REMOVE-L: Tüm saklı durumlardan bir normal dağılım çıkarır
- 4) ADD-L: Tüm saklı durumlara bir normal dağılım ekler

Yukarıdaki işlemler MSUMO'nun arama uzayını geniş olarak aramasını sağlamak için seçilmiştir. REMOVE-1 ve ADD-1 o anda arama uzayında bulunan durumun yakın komşuluğunu ararlar. Arama uzayında büyük atlamalar yapmak için, REMOVE-n ve ADD-n operatörlerini kullanabiliriz, ama bu çalışma bu işlemleri içermez. ADD-L ile yapıya yeni bir normal dağılım eklenir.

2.3.3. Hatasızlığın ve Karmaşıklığın Kontrolü: MultiTest Algoritması

SMM'in model karmaşıklığı SMM modelindeki saklı durum sayısı ile bir saklı durumdaki normal dağılım sayısının çarpımı olarak tanımlanır. Eşitlik olma durumunda, daha çok sayılı saklı durumlu yapıların daha karmaşık olduğu varsayılır çünkü bu SMM yapıları arasındaki bağlantıları arttırır. Bağlantı sayısı gibi, diğer karmaşıklık ölçütleri de kullanılabilir.

MSUMO'nun başında, ilk yapı en iyi yapı olarak seçilir. Aday modelleri oluşturmak için (C_i), işlemler karmaşıklarına göre sıralanarak uygulanır. Aday modeller oluşturulduktan sonra, bir aday ilk önce öğrenilir, sonra çapraz geçişleme ile gerçekleştirilir ve beklenen gerçekleme hatası tek-taraflı istatistiksel test (mesela k-katlı ikili t test) kullanılarak şu andaki en iyi ile karşılaştırılır.

Buradaki düşünce yapıyı mümkün olduğunca basit tutmaktır, ta ki ek karmaşıklık hatada anlamlı bir azaltma ile doğrulanmadığı sürece. Karmaşıklığı şu şekilde hesaba katıyoruz (YILDIZ VE ALPAYDIN, 2006): i j 'den daha kolay olmak üzere i ve j denen iki modeli karşılaştırdığımızda, model i 'nin beklenen hatasının model j 'nin beklenen hatasına eşit veya daha az olup olmadığını test ederiz:

$$H_0: \mu_i \leq \mu_j \text{ vs. } H_1: \mu_i > \mu_j$$

i modelini j modeline göre daha öne ayarlarız çünkü i daha basittir. Önsel bir sıralama olduğunu varsayarak, hipotez önseli takip eder ve tek taraflıdır. Eğer test reddetmezse, i 'yi tercih ederiz: ya $m_i < m_j$ iken, yani daha kolay olan model daha az hataya sahipse, onu seçeriz çünkü daha doğrudur; ya da $m_i = m_j$ iken daha kolay modeli seçeriz. Model j sadece test reddederse tercih edilir, mesela, ek karmaşıklık hatada anlamlı bir azalma ile doğrulanırsa, test verisi ilk tercihimizi değiştirir. Yani ilk olarak hatasızlığa bakılır, eşit hatasızlık durumunda daha kolay olan model tercih edilir.

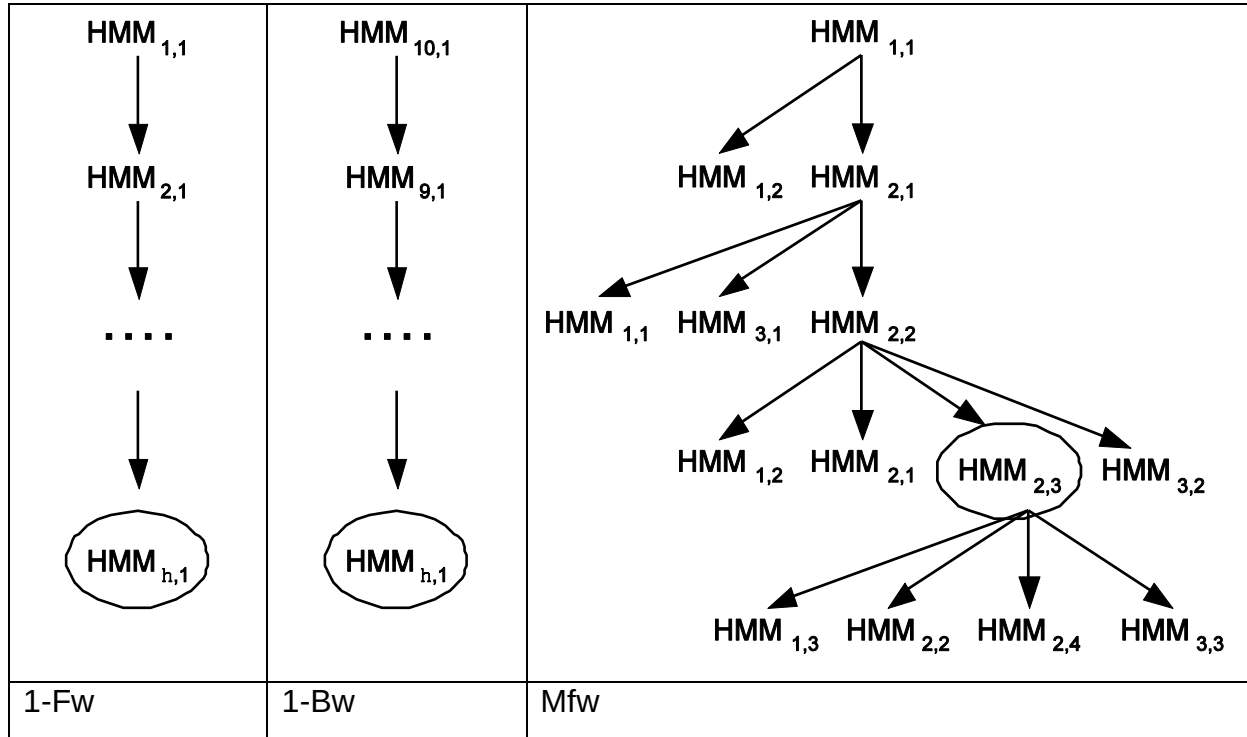
Böylelikle, MSUMO'ya göre, C_i adayı şu andaki en iyiden daha karmaşıksa, en iyi modelin yerine geçmek için anlamlı derecede daha az hataya sahip olması gerekir. En iyi model değişirse, algoritma yeni adaylar oluşturarak devam eder. En iyi model değiştirilemezse, algoritma durur ve en iyiyi döndürür.

Eğer herhangi bir anda işlemler uygulandığında ve genişlik-öncelikli arama yapmak istediğimizde, sonraki tüm durumları yaratırız ve yeni adaylar ile şu andaki en iyi arasından hangisinin daha iyi olduğunu bulmamız gerekir. Beklenen hata ile karmaşıklığı hesaba katarak ve istatistiksel test kullanarak iki modelden en iyisini seçebileceğimiz gibi MultiTest algoritması kullanarak herhangi bir sayıda modelden en iyisi de seçilebilir (YILDIZ VE ALPAYDIN, 2006).

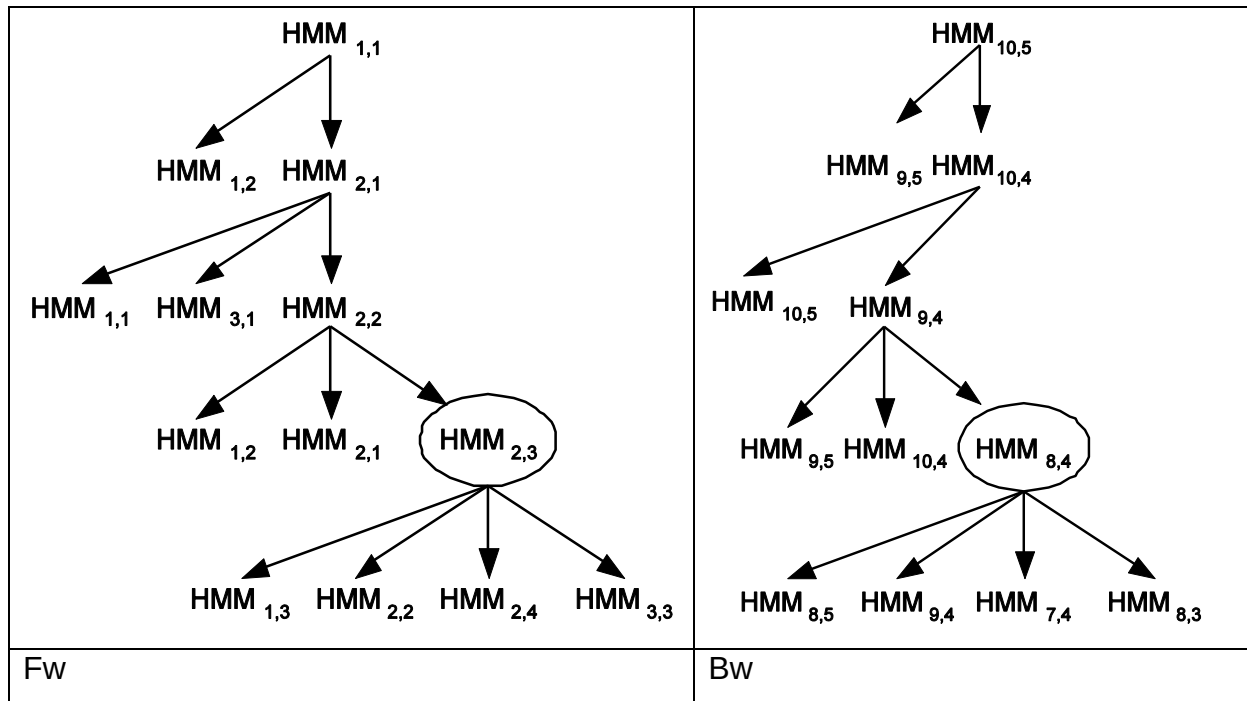
K modelden en iyisini seçmek için, MultiTest $K(K - 1) / 2$ test yapar. En iyi modelin güvenilirlik seviyesinin $(1-\alpha)$ olması için, her tek taraflı testin güvenilirlik seviyesi düzeltilmelidir. Kullanılabilecek düzeltme tekniklerinden ikisi Bonferroni ve Holmdur (HOLM, 1979).

2.3.4. 5 MSUMO Varyantı

Farklı arama algoritmaları MSUMOdaki tercihler değiştirilerek elde edilebilir. Arttırımlı veya azaltımlı bir algoritma, ilk durumu, işlem kümesini ve işlem deneme sırasını değiştirerek bulunabilir. Arama varyantını değiştirmek çözüm yapısının performansını etkiler, aramanın karmaşıklığını da çözüm bulunmadığı sürece etkiler. Biz aşağıdaki MSUMO varyantını inceliyoruz (Şekil 5 ve 6):



Şekil 5. 3 MSUMO varyantının durum uzayında örnek bir arama yapması. Mfw için, $HMM_{i,j}$ aşamasında dört aday model üretilir ($HMM_{i-1,j}$, $HMM_{i,j-1}$, $HMM_{i+1,j}$, $HMM_{i,j+1}$). 1-Fw'da aday modeller basitten karmaşığa doğru işlenirken 1-Bw'da karmaşıktan basite doğru işlenir.



Şekil 6. 2 MSUMO varyantının durum uzayında örnek bir arama yapması. Fw ve Bw için, $HMM_{i,j}$ aşamasında dört aday model üretilir ($HMM_{i-1,j}$, $HMM_{i,j-1}$, $HMM_{i+1,j}$, $HMM_{i,j+1}$). Fw'da aday modeller basitten karmaşığa doğru işlenirken Bw'da karmaşıktan basite doğru işlenir.

- 1) 1-adım ileri (1-Fw): İlk durumla ve tek normal dağılımla ile başlar ve iyileşme olmayana kadar ADD-1 kullanır.

- 2) 1-adım geri (1-Bw): $HMM_{10,1}$ ile başlar ve iyileşme olmayana kadar REMOVE-1 kullanır.
- 3) İleri MSUMO(Fw): Tek durum ve tek normal dağılım ile başlar ve yukarıdaki tüm işlemleri en basit olandan başlayarak derinlik-öncelikli artan sırada uygular.
- 4) Geri MSUMO(Bw): $HMM_{10,5}$ ile başlar ve tüm işlemleri en karmaşık olandan başlayarak derinlik-öncelikli azalan sırada uygular.
- 5) MultiTestli İleri MSUMO (Mfw): Tek normal dağılımlı tek durumla başlar ve tek tek uygulamak yerine, genişlik-öncelikli şekilde, tüm işlemleri bir kerede uygular ve MultiTest ile bu yeni adaylar ve şu andaki en iyi arasından yeni en iyi olanı seçer (örneğin, m işlem varsa, MultiTest $m+1$ 'in en iyisini seçer, yani m yeni aday ve şu andaki en iyi)

1-Fw ve 1-Bw arttırımlı ve azaltımlı temel algoritmalarıdır. Fw ve Bw çoklu işleme izin verir ve değişken aramayı kullanır, yani, ekleme ve çıkarmaya izin verir. Mfw tüm adayları bir durumda değerlendirerek genişlik-öncelikli arama yapar. Fw, Mfw ve 1-Fw arttırımlı olarak görülebilir ve Bw ve 1-Bw'de azaltımlı olarak görülebilir. MSUMO sisteminin genel olduğunu ve farklı ilk durumlar ve işlem kümeleri kullanarak MSUMO varyantlarına ulaşabileceğimizi belirtmek önemlidir.

3. DENEYLER VE BULGULAR

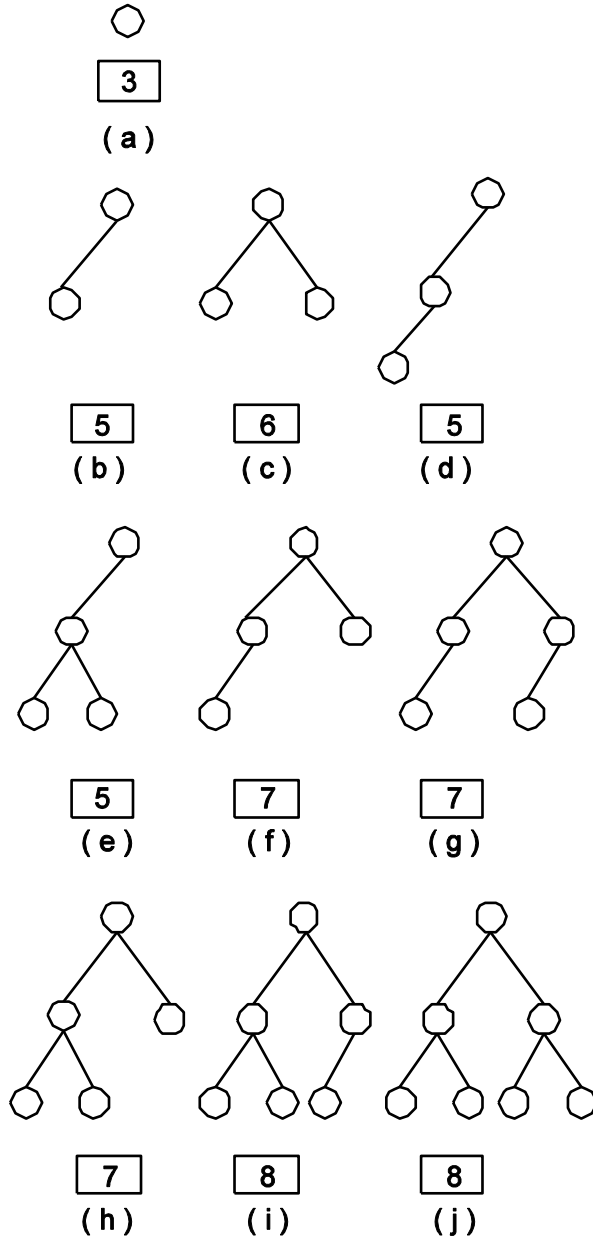
3.1. Karar Ağaçlarında VC-Boyut Hesaplama

3.1.1. Kapsamlı Arama Sonuçları

Hesaplama karmaşıklığı üssel arttığı gibi, karar ağacının derinliği de artarsa, olası tüm karar ağaçlarının sayısı da üssel artar. Bu nedenle, bu çalışmada derinliği 4'e kadar olan karar ağaçlarını tarandı. Sonra, karar ağacındaki düğüm sayısı, karar ağacının derinliği ve sol ile sağ alt ağaçların VC-boyutu cinsinden VC-boyutu tahmin eden özinelemeli bir formül bulmaya çalışıldı.

Belirli bir derinlikteki karar ağacı sayısını azaltmak için, iki benzer ağacın aynı VC-boyuta sahip olması gerektiği fikri kullanıldı. Bu nedenle, ilk olarak 1 derinlikli ağaçlar yaratıldı (1 tane ağaç var), sonra benzer ağaçlar yaratılmadığından emin

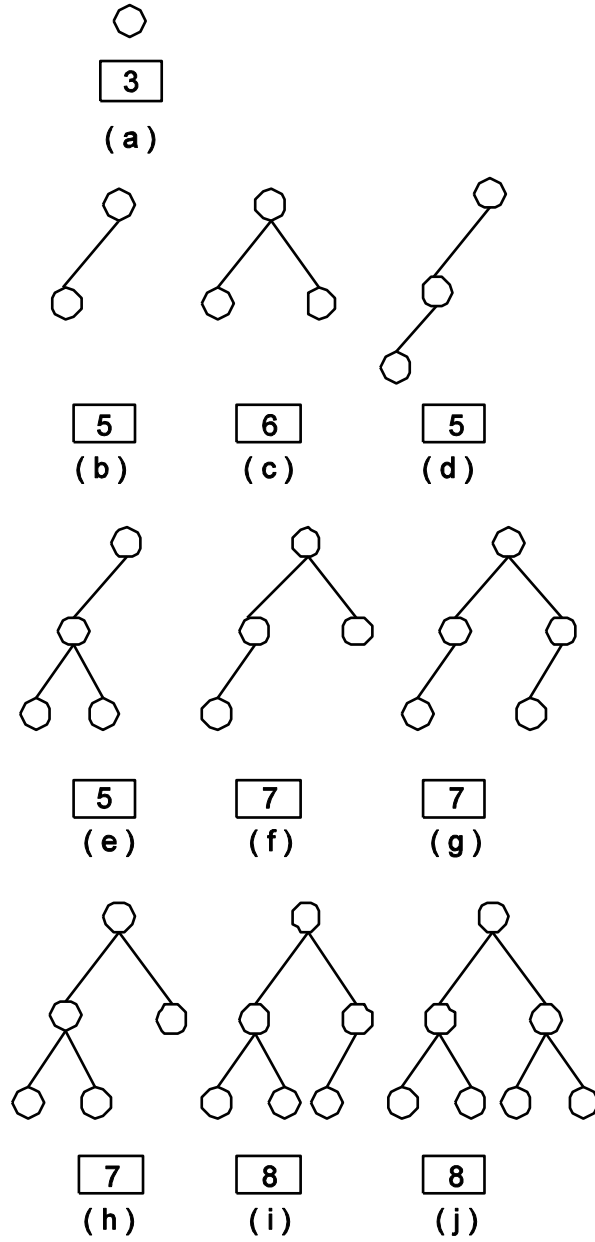
olarak ve önceki ağaçların son seviyesine (yapraklar) birer düğüm ekleyerek diğer ağaçlar yaratıldı.



Şekil 7. 3 boyutlu veri kümeleri için oluşturulan karar ağaçlarının VC-boyutları. Sadece iç düğümler gösterilmiştir.

Şekil 7 ve 8 veri kümeleri 3 ve 4 boyutlu karar ağaçlarının VC-boyutunu gösterir. Karar ağacındaki düğüm sayısı artarken VC-boyutun arttığı görülebilir ama karar ağacındaki düğüm sayısı artarken VC-boyutun sabit kaldığı istisnai durumlar var. Örneğin, Şekil 7'de Şekil 7a'dan Şekil 7b'ye giderken ve Şekil 7'te Şekil 7h'den Şekil 7i'ye giderken ve düğüm sayısı artarken VC-boyut artar. Diğer taraftan, düğüm

sayısı artarken, Şekil 7b'den Şekil 7c'ye giderken ve Şekil 5d den Şekil 5e ye giderken VC-boyut sabit kalır.



Şekil 8. 4 boyutlu veri kümeleri için oluşturulan karar ağaçlarının VC-boyutları. Sadece iç düğümler gösterilmiştir.

Karar ağacının derinliği artarken VC-boyutta artma gözlenir. Ancak bu bağımlılık çok açık değildir. Örneğin Şekil 7'de, derinlik 1'den derinlik 2'ye gidiş VC-boyutunda artmaya neden olsa da, bazı derinliği 2 olan karar ağaçları (örneğin, Şekil 7b) derinliği 3 olan karar ağaçları ile aynı VC-boyutuna sahiptir (Şekil 8d ve Şekil 8e). Şunu da görüyoruz ki derinliği 2 olan bir karar ağacının (Şekil 8c) VC-

boyutu derinliği 3 olan bir karar ağacının VC-boyutundan daha büyüktür (Şekil 8d ve Şekil 8e).

Karar ağacının VC-boyutu ile onun alt ağaçlarının VC-boyutu arasında bir ilişki yakalamaya çalıştığımızda bunun çok kolay olmadığını görürüz. Ama karar ağacının VC-boyutunun alt ağaçlarının VC-boyutundan büyük veya eşit olması gerektiği açıktır. Ana ağacın VC-boyutunu alt ağaçlarının VC-boyutu ile karşılaştırdığımız zaman, alt ağaç üst ağacın kullandığı özellik sayısından 1 azını kullanabilir. Örneğin, Şekil 8d'deki karar ağacının VC-boyutu Şekil 7b'de gösterilen alt ağacının VC-boyutuna eşit ya da ondan fazladır.

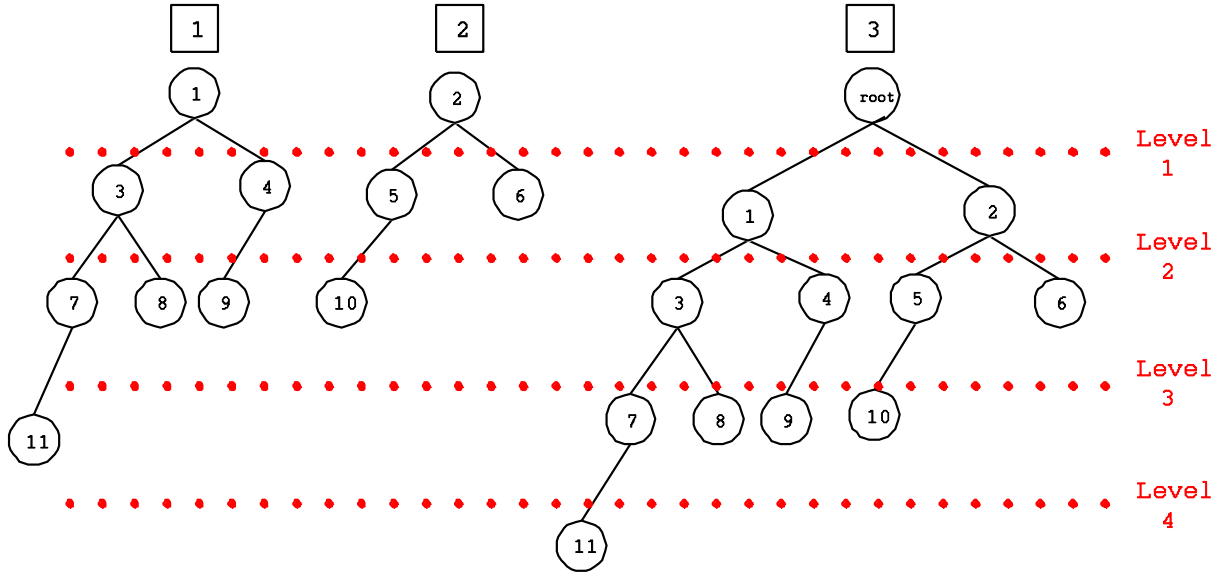
Özellik sayısı artarken ve / veya düğüm sayısı artarken, arama alanı üssel olarak artar. Bu dramatik artış $d > 4$ ve/veya karar ağacının derinliği 4'ten büyükken olmaya başlar. Daha büyük karar ağaçlarının VC-boyutunu hesaplamak için, mümkünse doğruluktan kaybetmeden arama alanını azaltmamız gerekir. Arama alanının üç boyutu vardır:

- 2^d farklı örnekten çekilen N veri noktalı olası tüm veri kombinasyonları
- N veri noktasının tüm işaretlendirmeleri
- Karar ağacı hipotez sınıfı H' 'deki tüm hipotezler

Tek bir veri kombinasyonu ile bir hipotez sınıfı H 'nin VC-boyutunun en az N olduğunu kanıtlamak mümkün olduğundan, arama alanını tüm veri kombinasyonlarının bir alt kümesini düşünerek azaltmaya karar verdik. Deneylerimizde, tüm $\binom{2^d}{N}$ veri kombinasyonu yerine, her karar ağacı yapısı için 1000 veri kombinasyonu alıyoruz.

Daha önce de açıkladığımız gibi, bu çalışmanın ana amacı bir karar ağacının VC- boyutunu onun sol ve sağ alt karar ağaçlarının VC- boyutuyla ilişkilendirmektir. Bu nedenle, sadece bir karar ağacının VC-boyutuna değil onun sol ve sağ alt karar ağaçlarının VC-boyutuna da ihtiyacımız var. Bunu elde etmek için, ters yönde çalışıyoruz ve VC-boyutu bilinen iki karar ağacını birleştiriyoruz ve sonra birleştirilmiş karar ağacının VC-boyutunu buluyoruz. Şekil 9 seviye seviye yapılan bu birleştirme işlemini gösterir. Böylelikle, birinci seviyeden başlayarak, önce düğüm 1 ve 2 seçilir ve yeni ağaca eklenir. Seviye 2 olarak adlandırdığımız sonraki seviyeden devam ederek, düğüm 3, 4, 5 ve 6 yeni karar ağacına eklenir. Üçüncü seviyede 7, 8, 9 ve 10

düğüm 11 yeni ağaca eklenir. Son olarak, dördüncü seviyede, düğüm 11 yeni ağaca eklenir.



Şekil 9. İki karar ağacını birleştirerek yeni bir karar ağacı oluşturma işlemi. Şekildeki sayılar birleştirilmiş karar ağacına düğümlerin eklenme sırasını gösterir.

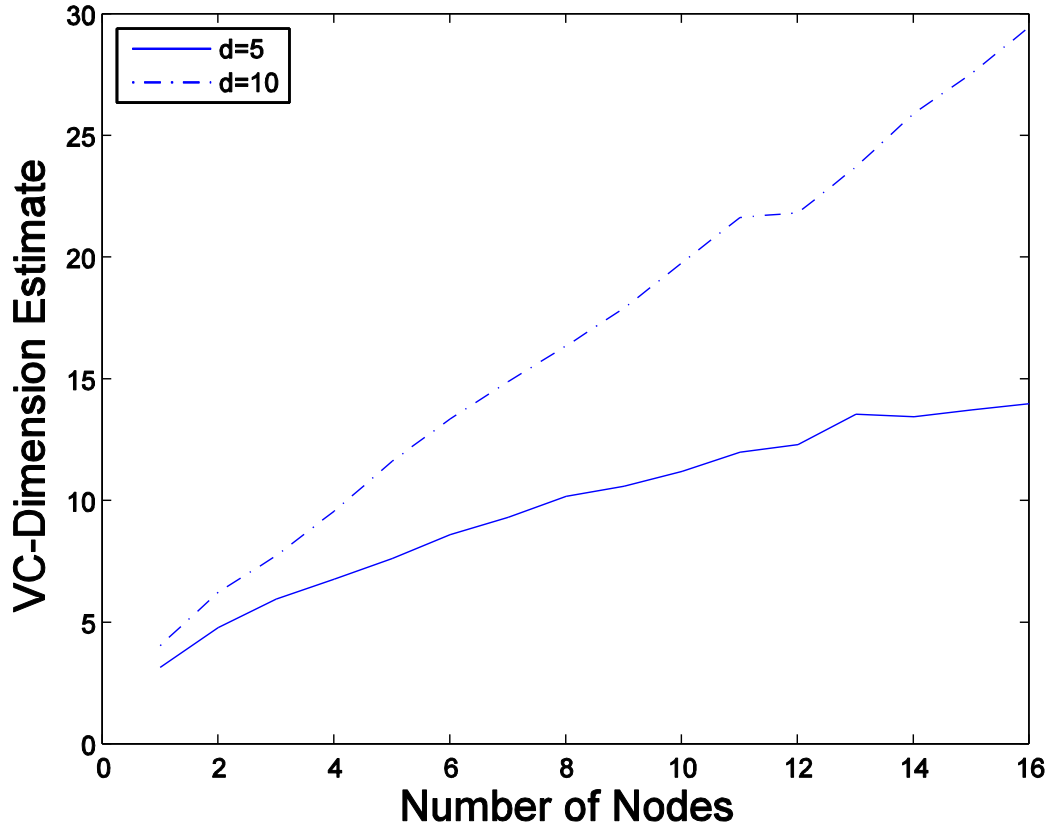
Bir karar ağacı kümesinin VC-boyutlarını bulduktan sonra, karar ağacının VC-boyutunu (i) sol ve sağ alt ağaçlarının VC-boyutuna (V_l, V_r) (ii) karar ağacındaki düğüm sayısına (M) (iii) karşılık gelen veri kümesindeki özellik sayısı (d) ile ilişkilendiren genel bir formül kestirmeye çalıştık. Bu verileri girdi olarak alan bir doğrusal regresyon kullandık ve onu 154 örnekli bir veri kümesi kullanarak eğittik. Sonuçlarımıza göre V_l ve V_r ile bir karar ağacının VC-boyutu arasındaki ilişki doğrusal gözüküyor. d ve M ile bir karar ağacının VC-boyutu arasındaki ilişki için de iki (d ve $\log d$) ve üç (M , $\sqrt{M}$, $\log M$) fonksiyon denedik. En küçük karelerle uydurma metodunu kullanarak, en iyi regresor

$$V = 0.608 + 1.008(V_l + V_r) + 0.366 \log d - 0.806 \log M$$

olarak bulundu, metodun $R^2 = 0.95$ 'tir, bu da oturtulan eğrinin iyi olduğunu gösterir.

3.1.2. Deney Dizaynı Sonuçları

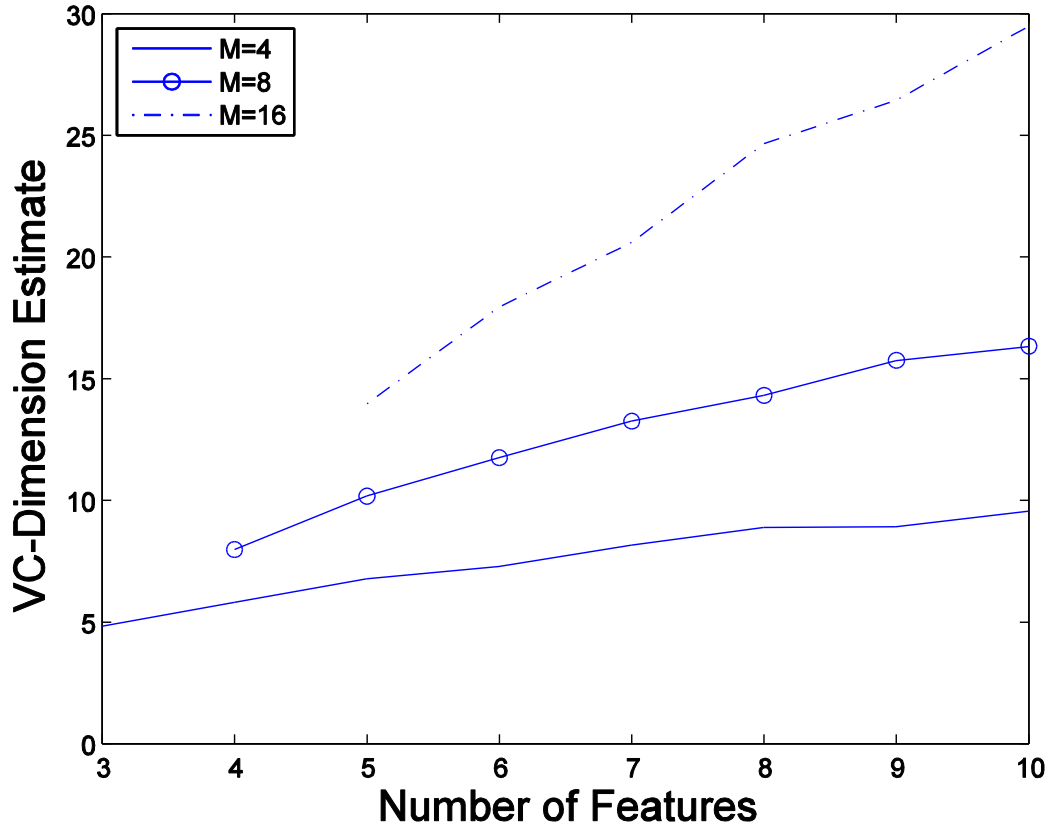
Bu deneylerde tek değişkenli karar ağacının VC-boyutunu optimize edilmiş deney dizaynı kullanarak kestiririz. Tüm durum alanını tarayabilmek için, özellik sayısı d olan M düğümlü karar ağaçlarının VC-boyutunu kestiriyoruz. (M 1'den 16'ya kadar ve d 1'den 10'a kadar değişmekte).



Şekil 10. Değişik özellik sayıları için ağaçtaki düğüm sayısına göre karar ağacının VC-boyutu.

VC-boyutunu kestirmek için iki parametre olduğu için (M ve d), karar ağacı yapılarının VC-boyutu kestirimlerini parametrelere göre çizdik. Şekil 10 tek değişkenli karar ağaçlarının VC-boyutu kestirimlerini farklı özellik sayıları (d) için ağaçtaki düğüm sayısına (M) göre gösterir. Karar ağacı VC-boyutunun özellik sayısının logaritmasıyla orantılı olduğunu görebiliriz.

Şekil 11 tek değişkenli karar ağaçlarının VC-boyutu kestirimlerini farklı düğüm sayıları (M) için veri kümesindeki özellik sayısına (d) göre gösterir. Bu şekil tek değişkenli karar ağaçlarının VC-boyutu kestirimlerinin M 'ye bağlı olduğunu gösterir. Her bir girdi ana düğümden yapraklara kadar bir kere kullanabileceği için, M 2^d den daha fazla olamaz.



Şekil 11. Değişik ağaç düğüm sayıları için veri kümesindeki özellik sayısına göre karar ağacının VC-boyutu.

Şekil 10 ve 11'in çizimlerini birleştirirince, tek değişkenli karar ağaçlarının VC-boyutu , u_1, u_2, u_3, u_4 sabit değerleri ile

$$u_1 \log(d - \log M)M + u_2 \log(d - \log M) + u_3 M + u_4$$

olarak yakınsanabileceğini anlarız.

Deneylerini ikinci fazında, doğrusal regresyon kullanarak bu sabit değerleri kestirmeye çalışıyoruz. u_1, u_2, u_3, u_4 'ü değişken olarak alınca ve tek değişkenli karar ağaçlarının VC-boyutu kestirimlerini (124 örnek) eğitim kümesi olarak kullanarak bulduğumuz kestirim fonksiyonu:

$$0.194 \log(d - \log M)M + 0.715 \log(d - \log M) + 1.012M + 0.673$$

'dir.

3.1.3. VC-Boyut Kestirimleri Kullanarak Karmaşıklık Kontrolü

Bu bölümde, kestirdiğimiz VC-boyutlarının ne kadar iyi kestirimler olduğunu test etmek için o kestirimleri karar ağaçlarında karmaşıklık kontrolünde kullanıyoruz. Karar ağaçlarında karmaşıklık kontrolü iki yolla yapılır. Bir düğüm için uygun olan

modeli seçerek (çok değişkenli karar ağaçları tek değişkenli, doğrusal veya doğrusal olmayan düğümler içerebilir (YILDIZ VE ALPAYDIN, 2001)) karar ağaçlarında karmaşıklık kontrolü yaparız veya budama kullanarak karar ağaçlarında karmaşıklık kontrolü yaparız. Bu çalışma sadece tek değişkenli karar ağaçlarını içerdiği için biz ikinci yaklaşımı kullanıyoruz ve önceki bölümde bulduğumuz VC-boyutu kestirimlerini budamada kullanıyoruz.

Arka budamada (Cvprune), alt karar ağacının performansını başka bir geçerleme veri kümesi üzerinde altağacı bir yaprakla değiştirerek hesaplıyoruz. Ezberleme varsa, daha karmaşık karar ağaçlarının gürültüyü öğrenmesini ve bir yapraktan daha kötü çalışmasını bekleriz. Burada geçerleme kümesi budama yaptıktan sonraki düğüm sayısını belirler ve düğüm sayısını önceden belirlemek mümkün değildir. Deneylerimizde, Cvprune için, veri kümesinin yüzde 20'si budama kümesi olarak ayrıldı.

Bir düğümü SRM kullanarak budamak için, öncelikle VC genel sınırını V VC-boyutu ve alt ağacın eğitim hatası E_t olmak üzere Formül 1'i kullanarak buluyoruz. Sonra, düğümü yaprağa dönüştürerek eğitim hatasını buluyoruz. Yaprığın VC boyutu 1 olduğu için, genel hatayı budanmış gibi bulabiliriz. Eğer yaprağın genel hatası alt ağacın genel hatasından az ise, alt ağacı buduyoruz, diğer durumda saklıyoruz. Eğer alt ağacın VC-boyutunu Formül 7'yi kullanarak bulursak (kapsamlı arama), bu çeşit bir budamayı SRMexhaust olarak adlandırıyoruz. Eğer VC-boyutunu Formül 8'i kullanarak bulursak (deney dizaynı), bu çeşit bir budamayı SRMexper olarak adlandırıyoruz. Bu iki budama çeşidinin ikisini de Cvprune ile karşılaştırıyoruz. Genelleme yapabilmek için, budama uygulanmamış ağaçların sonuçlarını da çalışmamızın içine koyduk (NOprune).

Karşılaştırmalarda şu üç tane fonksiyonu kullandık:

$$F_1 = x_0x_1 + x_0x_2 + x_1x_2$$

$$F_2 = x_0x_1 + x_0x_2 + x_0x_3 + x_1x_2 + x_1x_3 + x_2x_3$$

$$F_3 = x_0x_1' + x_0'x_1$$

Kıyas kriterlerimiz genel hata (5×2 çapraz geçerlemenin geçerleme katları üzerinde) ve karmaşıklık (karar ağacındaki toplam düğüm sayısı olarak ölçülür). Sentetik veri kümeleri yaratılmadan önce üç nokta üzerinde durmak gerekir:

- Özellik sayısı. Deneylerimizde $d = 8$ ve $d = 12$ yi kullandık, ekstra girdiler ağaçta olmaması gereken gereğinden fazla girdilerdir.

- Gürültü seviyesi. Orijinal girdiyi değiştirmek için 5 farklı gürültü seviyesi kullandık. ($\Psi = 0.0, 0.01, 0.05, 0.1, 0.2$).
- Eğitim veri kümesi büyüklüğü olası tüm kümeleri içeren 2^d örnekten seçildi. 4 çeşit veri alt kümesi yüzdesi kullandık. ($\tau = 10, 25, 50, 100$).

Tablo 1. 3 farklı fonksiyon için Noprune, CVprune, SRMExhaust ve SRMExper kullanılarak bulunan karar ağaçlarının hata oranları ile düğüm sayılarının ortalama ve standart sapmaları

Function	Hata Oranı			
	NoPrune	CVPrune	SRMExhaust	SRMExper
F_1	0.0± 0.0	0.0± 0.0	0.0± 0.0	0.0± 0.0
F_2	0.0± 0.0	0.0± 0.0	0.0± 0.0	0.0± 0.0
F_3	3.9± 2.8	8.5± 7.0	4.7± 3.6	4.9± 3.9
Function	Düğüm Sayısı			
	NoPrune	CVPrune	SRMExhaust	SRMExper
F_1	5.0± 0.0	5.0± 0.0	5.0± 0.0	5.0± 0.0
F_2	9.0± 0.0	9.0± 0.0	9.0± 0.0	9.0± 0.0
F_3	177.6± 115.8	83.3± 59.5	156.0± 97.5	155.2± 96.8

Tablo 2. Veri kümesindeki farklı özellik sayıları için Noprune, CVprune, SRMExhaust ve SRMExper kullanılarak bulunan karar ağaçlarının hata oranları ile düğüm sayılarının ortalama ve standart sapmaları

d	Hata Oranı			
	NoPrune	CVPrune	SRMExhaust	SRMExper
8	26.7± 3.4	32.3± 6.3	21.7± 4.9	21.4± 5.1
12	33.9± 1.9	33.0± 5.2	28.8± 2.0	35.9± 3.3
d	Düğüm Sayısı			
	NoPrune	CVPrune	SRMExhaust	SRMExper
8	23.2± 5.0	1.5± 2.2	5.8± 2.5	5.4± 2.5
12	413.0± 11.0	2.0± 2.1	3.4± 1.0	0.6± 1.0

Tablo 1, 3 farklı fonksiyon için Noprune, CVprune, SRMExhaust ve SRMExper kullanılarak bulunan karar ağaçlarının hata oranları ile düğüm sayılarının ortalama ve standart sapmalarını göstermektedir. Görüyoruz ki F_1 ve F_2 fonksiyonları 3 teknikte hiç hata vermeden çalıştı. Diğer yandan, dik olmayan ayrımlar gerektiren F_3 fonksiyonunu öğrenmek için, daha büyük ağaçlar üretilmek gerekmektedir. Bu

durumda, Cvprune ağaçları doğruluktan ödün vererek (yüzde 4.7'ye kıyasla 8.5) agresifçe budar (ağaç büyüklüğü 155'e kıyasla 83 düğüm) .

Tablo 2, veri kümesindeki farklı özellik sayıları için Noprune, CVprune, SRMexhaust ve SRMexper kullanılarak bulunan karar ağaçlarının hata oranları ile düğüm sayılarının ortalama ve standart sapmalarını göstermektedir. Tüm budama teknikleri karar ağaçlarının düğüm sayısını belirgin bir şekilde azaltmasına rağmen ($d = 8$ ve 25'ten 5'e düşen ağaç büyüklüğü ve $d = 12$ ve 413'ten 2'ye düşen ağaç büyüklüğü), sadece SRMexhaust aynı zamanda hata oranını iki özellik sayısı için de azaltabilmektedir.

Tablo 3. Farklı gürültü seviyeleri için Noprune, CVprune, SRMexhaust ve SRMexper kullanılarak bulunan karar ağaçlarının hata oranları ile düğüm sayılarının ortalama ve standart sapmaları

ψ	Hata Oranı			
	NoPrune	CVPrune	SRMexhaust	SRMexper
0.0	0.0± 0.0	0.0± 0.0	0.0± 0.0	0.0± 0.0
0.01	3.8± 0.4	1.4± 0.2	1.4± 0.2	1.4± 0.2
0.05	12.1± 0.5	5.4± 0.4	5.4± 0.5	5.4± 0.5
0.1	21.7± 0.8	10.4± 0.4	10.4± 0.4	28.4± 4.3
0.2	34.7± 1.1	27.9± 5.4	27.2± 6.2	35.7± 0.7
ψ	Düğüm Sayısı			
	NoPrune	CVPrune	SRMexhaust	SRMexper
0.0	5.0± 0.0	5.0± 0.0	5.0± 0.0	5.0± 0.0
0.01	135.5± 17.8	5.7± 2.2	5.0± 0.0	5.0± 0.0
0.05	361.2± 19.4	6.5± 3.2	5.0± 0.0	5.0± 0.0
0.1	579.2± 22.7	5.0± 0.0	5.0± 0.0	1.4± 0.8
0.2	846.1± 28.0	3.0± 1.3	3.2± 1.5	1.0± 0.0

Tablo 3, farklı gürültü seviyeleri için Noprune, CVprune, SRMexhaust ve SRMexper kullanılarak bulunan karar ağaçlarının hata oranları ile düğüm sayılarının ortalama ve standart sapmalarını göstermektedir. Beklenildiği gibi, gürültü seviyesi artırıldığında budama işe yaramaktadır. Örneğin gürültü seviyesi 0.01 gibi düşük olduğunda bile, Noprune, Cvprune, SRMexhaust ve SRMexper budama tekniklerine göre 30 kat daha büyük ağaçlar üretmektedir. Bir başka önemli nokta da, gürültü seviyesi arttıkça, budanmış ağaçların büyüklüklerinin neredeyse aynı kalmasıdır.

Yüksek gürültü oranları için SRMexper gereğinden fazla budamakta ve hata oranını arttırmaktadır.

Tablo 4. Farklı veri kümesi büyüklükleri için Noprune, CVprune, SRMexhaust ve SRMexper kullanılarak bulunan karar ağaçlarının hata oranları ile düğüm sayılarının ortalama ve standart sapmaları

T	Hata Oranı			
	NoPrune	CVPrune	SRMExhaust	SRMExper
100	16.5± 4.7	17.7± 14.2	12.0± 5.3	11.6± 5.2
50	15.4± 8.0	14.6± 17.0	11.7± 11.5	11.7± 11.5
25	29.3± 13.7	29.7± 21.1	28.6± 14.4	28.6± 14.4
10	30.0± 18.1	43.3± 19.2	34.2± 24.4	34.2± 24.4
T	Düğüm Sayısı			
	NoPrune	CVPrune	SRMExhaust	SRMExper
100	36.7± 11.8	7.8± 5.4	20.7± 15.4	21.1± 15.4
50	16.1± 6.0	4.1± 2.0	9.3± 8.7	9.3± 8.7
25	12.7± 3.7	3.5± 2.6	11.4± 4.0	11.4± 4.0
10	5.4± 1.3	1.4± 1.7	4.8± 2.1	4.8± 2.1

Tablo 4, farklı veri kümesi büyüklükleri için NOprune, CVprune, SRMexhaust ve SRMexper kullanılarak bulunan karar ağaçlarının hata oranları ile düğüm sayılarının ortalama ve standart sapmalarını göstermektedir. Bu durum için, örnekleme büyüklüğü azalırsa, ezberleme olur ve CV ve SRM budamaları çok işe yaramaz, hata oranları da örnekleme büyüklüğü azalırken artar.

3.1.4. L-dallı Ağaçlar

Şimdiye kadar, ikili özellikli tek değişkenli karar ağaçlarının VC-boyutuyla ilgilendik. Bu bölümde, düşüncemizi çoklu özellikli tek değişkenli karar ağaçlarını da katarak genişletiyoruz. Böyle bir veri kümesi için yaratılmış bir tek değişkenli karar ağacında, her bir iç düğümün kararın L tane farklı çıktısına göre L tane çocuğu (dal) olacaktır.

Bizim yaklaşımımız şu şekildedir: bazı kolay ağaçlar için, o ağaçların gerçek VC-boyutları bulmak için kapsamlı arama algoritması kullanırız. Verilen d özellikli ve her bir özelliği L değer alabilen bir veri kümesi için, olası tüm veri noktası sayısı L^d dir. Bu da VC-boyutu bulan algoritmanın karmaşıklığını

$$\sum_{N=1}^V \binom{L^d}{N} 2^{N|H|}$$

ya getirir.

Arama alanını azaltmak için, deneylerimizde, her bir karar ağacı yapısı için olası tüm $\binom{L^d}{N}$ veri kombinasyonları yerine 1000 tane rastgele örnek veri kombinasyonu alıyoruz. Bir örnek kombinasyonu bir karar ağacının VC-boyutunun N olduğunu göstermek için yeterlidir. Sonra, bu bulduğumuz değerlere herhangi bir karar ağacının VC-boyutunu kestirebilen için bir regressor oturtuyoruz. Karar ağacının VC-boyutunun genel formülü (I) alt ağaçlarının VC-boyutları $V_1, V_2, \dots, V_L$ (ii) ağaçtaki düğüm sayısı M ve (iii) ilgili veri kümesindeki özellik sayısına d bağlıdır.

$L = 3, 4, 5$ için en küçük karelere uydurma yöntemi kullanılarak bulunan en iyi regresörler:

$$V = -3.001 + 0.583 \sum_{i=1}^3 V_i + 2.531 \log d + 1.906 \log M$$

$$V = -1.629 + 0.556 \sum_{i=1}^4 V_i + 3.983 \log d - 0.407 \log M$$

$$V = 14.455 + 0.392 \sum_{i=1}^5 V_i - 4.769 \log d - 1.386 \log M$$

ve bu regresörlere karşılık gelen R^2 değerleri sırasıyla 0.91, 0.86 ve 0.78 dir. R^2 in L artarken azalması, varsaydığımız basit arttırımlı doğrusal modelin daha karmaşık ağaçlar için geçerli olamayabileceğini gösterir. Şu nedenle de olabilir: karar ağacının karmaşıklığı artarken (daha fazla L için), daha fazla veriye ihtiyacımız olur ve örnek olarak aldığımız 1000 veri yeterli olmayabilir. Hala, az L ile de (ve genelde karar ağaçlarında L nadir olarak 5'ten büyüktür), ikili özellikli ağaçlarda yaptığımız gibi karmaşıklık kontrolünde kullanılabilecek kadar iyi bir model öğrendiğimizi düşünebiliriz.

3.2. Saklı Markov Modellerinde Model Seçimi

3.2.1. Deney Faktörleri ve Değerlendirme Kriterleri

Saklı Markov Modelleri ile ilgili deneylerde, şu faktörleri inceliyoruz:

- *Aramada kullanılan MSUMO varyantı.* Bunlar ileri aramaya karşılık geri aramayı gerçekleyen, tek operatöre karşın çoklu operatörü gerçekleyen ve

derinlik öncelikliye karşın genişlikli-öncelikli arama yapan 1-Fw, 1-Bw, Fw, Bw ve Mfw'dir.

- *Karşılaştırma kullanarak istatistiksel test.* K-katlı ikili t testi, parametrelili test olarak kullanıyoruz. Diğer parametrelili/parametresiz testler de kullanılabilir.
- *Testin güvenilirlik seviyesi(1- α).* 0.95,0.99, 0.995 ve 0.999 u kullanıyoruz ama sadece 0.95'in sonucunu raporluyoruz.
- *Çoklu test uygularken kullanılan düzeltme.* Holm düzeltmesi kullanıyoruz (HOLM, 1979) ve düzeltme olmayan ile istatistiksel test kullanılmayan sonuçlar ile karşılaştırıyoruz.

SMM'leri Kevin Murphy tarafından yazılan SMM kütüphanesi kullanılarak eğitildi. Yapının yeniden eğitilmesi, bir işlem uygulandığında, olasılıklara tekrar değer atayarak ve baştan tekrar eğiterek yapılır. Hiç bir olasılık sabit tutulmaz veya dondurulmaz.

MSUMO varyantları tarafından bulunan yapılar optimum yapı ile karşılaştırılır. Her bir veri kümesinin optimum yapısı tek saklı durumlu SMM'den 10 saklı durumlu SMM'lere kadar ve her durumda tek normal dağılımdan her durumda 5 normal dağılıma kadar tüm SMM'lere kapsamlı arama uygulayarak bulunur. Bu nedenle, içinden seçebileceğimiz $10 \times 5 = 50$ farklı yapı vardır.

İstatistiksel test kullanarak, her bir 50 yapıyı diğer 49 yapı ile karşılaştırıyoruz ve MultiTest kullanarak optimum yapıyı buluyoruz. (MultiTest bu modellerin sıralamasını verir ve bu sıralamayı MSUMO'nun bulduğu yapı ile ve optimal yapının arasındaki "mesafe" yi hesaplamak için kullanırız; aşağıda verilmiştir).

5 MSUMO varyantını kullanarak bulunan yapıları 3 kriter açısından karşılaştırıyoruz:

- 1) Kestirilmiş yapının hata oranı
- 2) Kestirilmiş yapının karmaşıklığı ve
- 3) Yapı bulunana kadar yapılan aramanın hesaplama karmaşıklığı

Bulunan yapının iyiliğini ölçmek için, MultiTest tarafından bulunan (her bir veri kümesi için) ve optimum yapısı birinci pozisyonda, ikinci en iyi yapısı ikinci pozisyonda, en kötü yapısı son pozisyonda olan *sıralama* ölçütü tanımlarız. Hata oranı ve karmaşıklık MultiTest tarafından bulunan sırayı bulmak için kullanılır: Düşük sıra hata oranı açısından optimum yapıya yakın bir yapı bulduğumuza veya

hata oranı karşılaştırılabilirse yüksek sıralı bir yapıdan daha kolay bir yapı bulduğumuza işaret eder.

İkinci ölçüt *derece* (rank) son duruma ne kadar hızlı gideceğimizi belirler ve durum sayısını aramanın karmaşıklığı olarak kullanır. Bu ölçütü düşük tutmak önemlidir çünkü bu eğitilmesi ve gerçekleşmesi gereken yapıların sayısına denk gelir (çoklu katlar üzerinden). Algoritmalar kullanılarak bulunan yapılar sıralarını belirlemek için MultiTest kullanılarak sıralanır ve sonra optimum yapıya yakınlıklarına göre derecelenirler. 2 veya daha fazla MSUMO varyantı aynı yapıyı kullanıyorsa arama karmaşıklığı hesaba katılır. Bu durumda, derecelmeler sıralamanın yerini alır ve daha az durum ziyaret eden daha az dereceyi alır.

Tablo 5. Derecelerin hesaplanmasına bir örnek.

	Fw	Bw	MFw	1-Fw	1-Bw
Sıralama	3	2	1	1	2
Durum sayısı	10	5	3	4	5
Derece	5	3.5	1	2	3.5

Ziyaret edilen durumlar da eşitse, derecelerin ortalaması her bir yapıya verilir. Dereceleri hesaplama örneği Tablo 5'te verilmiştir. Düşük dereceli MSUMO varyantı optimum yapıya yakın bir yapı bulunduğunu işaret eder ve bunu yaparken daha yüksek dereceli MSUMO ile karşılaştırıldığında arama uzayında daha az durumu ziyaret eder.

Tablo 6. Veri kümelerinin özellikleri.

	Kısaltma	Büyüklik	İşaret Sayısı	Özellik Sayısı
IDIAP	twoh	490	7	20
eNTERFACE	eface	760	19	32
British	bsl	980	91	22
Australian	au	6650	95	8

3.2.2. Veri Kümeleri

Veri kümelerinin özellikleri Tablo 6'da görülebilir. Özellikler her bir çerçevedeki frame sayısını gösterir. IDIAP veri kümesi üzerinde özellik ayıklama hakkında daha fazla bilgi (MARCEL VE JUST) de bulunabilir. ENTERFACE'06 (ARAN VE DİĞERLERİ, 2009) veri kümesinde, soldan-sağa model uyguladığımızda, maximum 5 durumlu SMM eğitebildik, bu yüzden seçim $5 \times 5 = 25$ yapı ile yapıldı. İngilizce

İşaret Dili veri kümesinde (BOWDEN), soldan -sağa ve soldan-sağa-döngü modelleri uyguladığımızda, maximum 8 durumlu SMM eğitebildik, bu yüzden seçimi $8 \times 5 = 40$ yapı kullanarak yaptık. Bu çalışmada, 4 tane Avustralya veri kümesinin (ASUNCION VE NEWMAN, 2007) alt kümesini kullandık: ilk 80, 60, 40, 20 işareti içeren ve sırayla isimleri au80, au60, au40, au20 olarak verilmiştir.

Tablo 7. MSUMO varyantlarının istatistiksel test kullanmadan Derece(Sıra)'ları.

Veri kümesi	Model	MFw	Fw-1	Bw-1	Fw	Bw
bsl	full	2.5(1,3)	1.0(1,2)	4.0(5,2)	2.5(1,3)	5.0(31,12)
bsl	lr	2.5(1,3)	1.0(1,2)	4.0(5,2)	2.5(1,3)	5.0(25,8)
bsl	lrl	2.5(1,3)	1.0(1,2)	5.0(21,4)	2.5(1,3)	4.0(3,17)
eface	full	4.5(43,3)	3.0(43,2)	2.0(18,2)	4.5(43,3)	1.0(10,5)
eface	lr	3.5(16,3)	2.0(16,2)	5.0(21,3)	3.5(16,3)	1.0(3,3)
eface	lrl	2.5(20,7)	5.0(46,2)	4.0(44,4)	2.5(20,7)	1.0(9,3)
twoh	full	3.0(35,9)	4.0(46,5)	5.0(48,2)	2.0(8,13)	1.0(2,3)
twoh	lr	2.5(6,12)	4.0(41,5)	5.0(47,3)	2.5(6,12)	1.0(2,5)
twoh	lrl	3.5(13,3)	2.0(13,2)	5.0(38,2)	3.5(13,3)	1.0(4,4)
au20	full	2.0(2,28)	5.0(48,3)	4.0(44,2)	3.0(16,14)	1.0(2,4)
au20	lr	3.0(16,15)	5.0(47,8)	4.0(42,2)	2.0(16,11)	1.0(1,4)
au20	lrl	3.0(4,10)	5.0(49,4)	4.0(45,2)	2.0(4,6)	1.0(1,5)
au40	full	1.0(1,26)	5.0(43,7)	4.0(42,2)	3.0(2,26)	2.0(2,3)
au40	lr	3.0(7,21)	5.0(44,10)	4.0(44,2)	2.0(7,18)	1.0(1,3)
au40	lrl	3.0(2,12)	5.0(47,4)	4.0(45,2)	2.0(2,9)	1.0(1,5)
au60	full	2.0(2,26)	5.0(42,10)	4.0(42,2)	3.0(3,24)	1.0(1,3)
au60	lr	2.0(1,27)	5.0(49,2)	4.0(34,3)	3.0(4,21)	1.0(1,4)
au60	lrl	3.0(4,12)	4.0(46,4)	5.0(47,2)	2.0(4,9)	1.0(2,5)
au80	full	3.0(1,27)	5.0(49,2)	4.0(41,3)	2.0(1,25)	1.0(1,3)
au80	lr	3.0(1,27)	5.0(49,2)	4.0(36,3)	2.0(1,22)	1.0(1,4)
au80	lrl	3.0(4,10)	4.0(46,4)	5.0(47,2)	2.0(4,6)	1.0(3,5)
	Ortalama	2.8(9,14)	3.9(37,4)	4.2(36,2)	2.6(8,12)	1.6(5,5)

Tablo 8. MSUMO varyantlarının istatistiksel test kullanmadan Ortalama Hata(Durum Sayısı, Karışım Sayısı)'ları.

Veri Kümesi	Model	MFw	Fw-1	Bw-1	Fw	Bw	En İyi
bsl	full	7.0(1,1)	7.0(1,1)	11.7(10,1)	7.0(1,1)	13.6(6,4)	7.0(1,1)
bsl	lr	7.0(1,1)	7.0(1,1)	10.2(8,1)	7.0(1,1)	12.9(4,5)	7.0(1,1)
bsl	lrl	9.0(1,1)	9.0(1,1)	9.5(6,1)	9.0(1,1)	9.5(6,1)	9.0(1,1)
eface	full	8.0(1,1)	8.0(1,1)	4.3(10,1)	8.0(1,1)	5.8(10,4)	4.3(10,1)
eface	lr	8.0(1,1)	8.0(1,1)	7.8(4,1)	8.0(1,1)	6.7(5,5)	6.5(4,4)
eface	lrl	6.6(2,2)	7.9(1,1)	5.4(8,1)	6.6(2,2)	6.2(10,5)	5.1(6,4)
twoh	full	2.7(2,3)	2.9(4,1)	2.0(10,1)	1.8(4,4)	1.4(10,5)	1.2(9,4)
twoh	lr	1.9(3,3)	2.2(4,1)	2.7(9,1)	1.8(3,3)	1.2(8,5)	1.0(6,5)
twoh	lrl	1.9(1,1)	1.8(1,1)	2.2(10,1)	1.8(1,1)	1.6(9,5)	1.4(1,4)
au20	full	34.1(9,5)	55.5(2,1)	42.0(10,1)	39.2(5,5)	34.1(9,5)	34.0(10,4)
au20	lr	39.5(3,5)	46.2(7,1)	43.9(10,1)	39.5(3,5)	33.1(9,5)	33.1(9,5)
au20	lrl	33.7(1,5)	46.4(3,1)	46.3(10,1)	33.7(1,5)	32.8(8,5)	32.8(8,5)
au40	full	44.0(8,5)	56.5(6,1)	53.5(10,1)	44.3(10,5)	44.3(10,5)	44.0(8,5)
au40	lr	45.1(7,5)	55.7(10,1)	55.7(10,1)	45.1(7,5)	42.8(10,5)	42.8(10,5)
au40	lrl	43.4(2,5)	55.4(3,1)	55.5(10,1)	43.4(2,5)	43.1(8,5)	43.1(8,5)
au60	full	47.6(8,5)	56.4(10,1)	56.3(10,1)	47.8(9,4)	47.1(10,5)	47.1(10,5)
au60	lr	46.4(9,5)	70.5(1,1)	59.0(9,1)	48.3(8,4)	46.4(9,5)	46.4(9,5)
au60	lrl	47.2(2,5)	60.0(3,1)	59.7(10,1)	47.2(2,5)	46.8(8,5)	46.7(4,5)
au80	full	52.9(10,5)	74.8(1,1)	61.7(9,1)	52.9(10,5)	52.9(10,5)	52.9(10,5)
au80	lr	52.6(9,5)	74.8(1,1)	64.8(9,1)	52.6(9,5)	52.6(9,5)	52.6(9,5)
au80	lrl	53.1(1,5)	65.0(3,1)	65.6(10,1)	53.1(1,5)	53.0(8,5)	52.5(4,5)

3.2.3. Bulgular

Hiç istatistiksel test kullanmadan bulunan dereceleme ve sıralamaları Tablo 7' de görebiliriz. İlk sayılar dereceleri gösterir ve parantez içindeki sayılar sırasıyla sıralamayı ve her bir algoritma tarafından ziyaret edilen durum sayısını gösterir. Koyu

yazılmış bilgiler MultiTest (sıra=1)'e göre algoritmanın en iyi modeli bulduğu anlamına gelir. Birden çok algoritmanın optimumu bulduğu veri kümeleri olmasına rağmen, dereceleri ziyaret ettikleri durum sayısı nedeniyle değişiktir. Optimum (BEST) tüm durumlar üzerinde MultiTest uygulayarak bulunur, bu nedenle BEST için, sıra = 1, ziyaret edilen durum sayısı toplam yapı sayısıdır. Bu sonuçlara göre, algoritmalara baktığımızda, en iyi algoritmanın (dereceye göre) Bw olduğunu görüyoruz. Hiçbir istatistiksel test kullanmadığımızda, iki algoritmayı ortalama hatalarına göre karşılaştırırız. En düşük hatalı algoritma bir sonraki aday olarak seçilir. Avustralya veri kümesi için karışım sayısı ve durum sayısı artarken hata çoğunlukla azaldığı için, en iyi algoritmalar genellikle çok sayıda karışım ve durum sayısına sahiptir. Bu Bw algoritmalarına MSUMO varyantlarına kıyasla avantaj verir, dolayısıyla o en iyi algoritmadır. Beklenildiği gibi sıralamaya göre Bw en iyidir. Ziyaret edilen durum sayılarını karşılaştırdığımızda, derecesi en kötü olmasına rağmen, en iyi algoritmanın 1-Bw olduğunu görürüz. 1-Fw daha az durum ziyaret etmektedir ama BW kadar başarılı değildir.

MSUMO varyantlarının buldukları yapılar ve beklenen hata oranları Tablo 8'de görülebilir. İlk sayı beklenen hata oranıdır ve parantez içindeki sayılar durum sayısını ve her bir algoritma tarafından bulunan yapının karışım sayısını gösterir. Avustralya veri kümesinde, Bw'nin optimum yapıyı veya optimuma yakın olan yapıları bulduğunu görürüz. Ayrıca Bw, Fw ve Mfw nin en düşük sıralamaya (optimuma yakın) sahip olduğunu görüyoruz, bu da işlemlerin iki yönde de uygulanmasını destekler.

Tablo 9. MSUMO varyantlarının k-katlı t testi uygulanıp düzeltme uygulanmaksızın elde edilen Derece(Sıra)'ları.

Veri kümesi	Model	MFw	Fw-1	Bw-1	Fw	Bw
bsl	full	2.5(1,3)	1.0(1,2)	4.0(1,10)	2.5(1,3)	5.0(1,22)
bsl	lr	2.5(1,3)	1.0(1,2)	4.0(1,8)	2.5(1,3)	5.0(1,20)
bsl	lrl	2.5(1,3)	1.0(1,2)	5.0(10,6)	2.5(1,3)	4.0(1,20)
eface	full	3.5(40,3)	2.0(40,2)	5.0(40,10)	3.5(40,3)	1.0(9,20)
eface	lr	2.5(1,3)	1.0(1,2)	4.0(1,5)	2.5(1,3)	5.0(1,15)
eface	lrl	4.5(41,3)	3.0(41,2)	1.0(22,4)	4.5(41,3)	2.0(25,16)
twoh	full	3.5(50,3)	2.0(50,2)	5.0(50,10)	3.5(50,3)	1.0(15,18)
twoh	lr	4.5(48,3)	3.0(48,2)	2.0(6,8)	4.5(48,3)	1.0(4,7)
twoh	lrl	2.5(1,3)	1.0(1,2)	4.0(1,10)	2.5(1,3)	5.0(1,24)
au20	full	3.0(39,10)	5.0(48,2)	4.0(43,6)	2.0(21,12)	1.0(16,14)
au20	lr	3.0(42,7)	4.0(47,2)	5.0(50,5)	2.0(42,5)	1.0(28,14)
au20	lrl	2.0(24,7)	5.0(45,2)	4.0(41,7)	1.0(24,5)	3.0(24,21)
au40	full	3.0(18,15)	5.0(50,2)	4.0(43,6)	2.0(18,10)	1.0(4,5)
au40	lr	4.5(46,3)	3.0(46,2)	2.0(32,3)	4.5(46,3)	1.0(22,14)
au40	lrl	2.0(1,9)	4.0(41,2)	5.0(41,10)	1.0(1,6)	3.0(1,18)
au60	full	4.5(47,3)	3.0(47,2)	2.0(44,4)	4.5(47,3)	1.0(7,9)
au60	lr	4.5(45,3)	3.0(45,2)	2.0(32,3)	4.5(45,3)	1.0(17,10)
au60	lrl	2.0(1,9)	4.0(41,2)	5.0(41,10)	1.0(1,6)	3.0(1,19)
au80	full	4.5(47,3)	3.0(47,2)	2.0(38,3)	4.5(47,3)	1.0(12,9)
au80	lr	4.5(45,3)	3.0(45,2)	2.0(35,3)	4.5(45,3)	1.0(2,4)
au80	lrl	2.0(1,10)	4.0(41,2)	5.0(41,10)	1.0(1,6)	3.0(4,11)
	Ortalama	3.2(26,5)	2.9(35,2)	3.6(29,7)	2.9(25,4)	2.3(9,15)

Tablo 10. MSUMO varyantlarının k-katlı t testi uygulanıp düzeltme uygulanmaksızın elde edilen Ortalama Hata (Durum Sayısı, Karışım Sayısı)'ları.

Veri Kümesi	Model	MFw	Fw-1	Bw-1	Fw	Bw	En İyi
bsl	full	7.0(1,1)	7.0(1,1)	7.0(1,1)	7.0(1,1)	7.0(1,1)	7.0(1,1)
bsl	lr	7.0(1,1)	7.0(1,1)	7.0(1,1)	7.0(1,1)	7.0(1,1)	7.0(1,1)
bsl	lrl	9.0(1,1)	9.0(1,1)	9.0(4,1)	9.0(1,1)	9.0(1,1)	9.0(1,1)
eface	full	8.0(1,1)	8.0(1,1)	8.0(1,1)	8.0(1,1)	6.3(2,3)	5.0(9,1)
eface	lr	8.0(1,1)	8.0(1,1)	8.0(1,1)	8.0(1,1)	8.0(1,1)	8.0(1,1)
eface	lrl	7.9(1,1)	7.9(1,1)	5.4(8,1)	7.9(1,1)	6.1(3,4)	6.6(3,1)
twoh	full	4.9(1,1)	4.9(1,1)	4.9(1,1)	4.9(1,1)	1.8(4,4)	2.8(3,1)
twoh	lr	4.9(1,1)	4.9(1,1)	2.2(4,1)	4.9(1,1)	1.0(6,5)	1.8(3,3)
twoh	lrl	1.8(1,1)	1.8(1,1)	1.8(1,1)	1.8(1,1)	1.8(1,1)	1.8(1,1)
au20	full	48.2(1,5)	57.3(1,1)	46.2(6,1)	41.7(3,4)	39.1(6,3)	35.6(8,4)
au20	lr	51.4(1,3)	57.3(1,1)	46.2(7,1)	51.4(1,3)	42.2(4,3)	36.0(6,5)
au20	lrl	36.5(1,3)	49.8(1,1)	46.5(5,1)	36.5(1,3)	36.5(1,3)	34.2(1,4)
au40	full	50.0(3,5)	67.2(1,1)	56.5(6,1)	50.0(3,5)	44.0(8,5)	46.0(7,4)
au40	lr	67.2(1,1)	67.2(1,1)	55.9(9,1)	67.2(1,1)	55.0(4,3)	44.3(8,4)
au40	lrl	43.9(1,4)	57.5(1,1)	57.5(1,1)	43.9(1,4)	43.9(1,4)	43.9(1,4)
au60	full	70.5(1,1)	70.5(1,1)	58.5(8,1)	70.5(1,1)	49.8(7,4)	47.8(9,4)
au60	lr	70.5(1,1)	70.5(1,1)	59.0(9,1)	70.5(1,1)	53.8(4,5)	48.3(8,4)
au60	lrl	47.5(1,4)	61.2(1,1)	61.2(1,1)	47.5(1,4)	47.5(1,4)	47.5(1,4)
au80	full	74.8(1,1)	74.8(1,1)	61.7(9,1)	74.8(1,1)	56.1(7,4)	54.1(9,4)
au80	lr	74.8(1,1)	74.8(1,1)	64.8(9,1)	74.8(1,1)	52.6(9,5)	53.9(10,4)
au80	lrl	53.1(1,5)	66.4(1,1)	66.4(1,1)	53.1(1,5)	52.5(4,5)	53.1(1,5)

Tablo 9 ve 10'da düzeltme olmayan sonuçları görebiliriz. Bu veri kümelerini kullanarak ve düzeltme kullanmayarak Bw'nin hala derece ve sıralamaya göre en iyi olduğunu görebiliriz. Algoritmaları karşılaştırırken bir istatistiksel test kullandığımız için, karmaşık bir algoritmanın daha kolay bir algoritmayı yenmesi daha zor oluyor, bu da istatistiksel test kullanmamaya kıyasla Bw'deki derece azalmasını açıklar. Bu durumda, ziyaret edilen durum sayısını düşünüldüğünde, ileri algoritmalar geri algoritmalarından daha iyidir, 1-Fw en iyisidir. Bw en çok sayıda durumu ziyaret eder. Bw sıralamaya göre hala en iyidir ve bir sonraki en iyiye göre 15 algoritma daha

iyidir. İstatistiksel test uyguladığımızda arttırımlı algoritmaların hata oranının hızlı arttığını da görürüz.

Tablo 11. MSUMO varyantlarının k-katlı t testi ile Holm düzeltmesi uygulanarak elde edilen Derece(Sıra)'ları.

Veri kümesi	Model	MFw	Fw-1	Bw-1	Fw	Bw
bsl	full	2.5(1,3)	1.0(1,2)	4.0(1,10)	2.5(1,3)	5.0(1,22)
bsl	lr	2.5(1,3)	1.0(1,2)	4.0(1,8)	2.5(1,3)	5.0(1,20)
bsl	lrl	2.5(1,3)	1.0(1,2)	5.0(4,6)	2.5(1,3)	4.0(1,20)
eface	full	2.5(1,3)	1.0(1,2)	4.0(1,10)	2.5(1,3)	5.0(9,20)
eface	lr	2.5(1,3)	1.0(1,2)	4.0(1,5)	2.5(1,3)	5.0(1,15)
eface	lrl	2.5(1,3)	1.0(1,2)	4.0(8,4)	2.5(1,3)	5.0(23,16)
twoh	full	2.5(1,3)	1.0(1,2)	4.0(1,10)	2.5(1,3)	5.0(29,18)
twoh	lr	2.5(1,3)	1.0(1,2)	4.0(4,8)	2.5(1,3)	5.0(42,7)
twoh	lrl	2.5(1,3)	1.0(1,2)	4.0(1,10)	2.5(1,3)	5.0(1,24)
au20	full	5.0(48,5)	4.0(47,2)	3.0(44,6)	2.0(24,12)	1.0(3,14)
au20	lr	4.0(44,3)	3.0(44,2)	5.0(47,5)	2.0(39,5)	1.0(15,14)
au20	lrl	2.0(1,7)	4.0(38,2)	5.0(39,7)	1.0(1,5)	3.0(1,21)
au40	full	3.0(28,15)	5.0(50,2)	4.0(45,6)	2.0(28,10)	1.0(12,5)
au40	lr	4.5(50,3)	3.0(50,2)	2.0(36,3)	4.5(50,3)	1.0(19,14)
au40	lrl	2.0(2,9)	4.0(41,2)	5.0(41,10)	1.0(2,6)	3.0(2,18)
au60	full	4.5(49,3)	3.0(49,2)	2.0(47,4)	4.5(49,3)	1.0(5,9)
au60	lr	4.5(48,3)	3.0(48,2)	2.0(38,3)	4.5(48,3)	1.0(23,10)
au60	lrl	2.0(1,9)	4.0(37,2)	5.0(37,10)	1.0(1,6)	3.0(1,19)
au80	full	4.5(50,3)	3.0(50,2)	2.0(40,3)	4.5(50,3)	1.0(4,9)
au80	lr	4.5(49,3)	3.0(49,2)	2.0(32,3)	4.5(49,3)	1.0(11,4)
au80	lrl	1.0(1,9)	4.0(37,2)	5.0(37,10)	2.0(2,6)	3.0(11,11)
	Ortalama	3.0(18,5)	2.5(26,2)	3.8(24,7)	2.7(17,4)	3.0(10,15)

Tablo 12. MSUMO varyantlarının k-katlı t testi ile Holm düzeltmesi uygulanarak elde edilen Ortalama Hata (Durum Sayısı, Karışım Sayısı)'ları.

Veri Kümesi	Model	MFw	Fw-1	Bw-1	Fw	Bw	En İyi
bsl	full	7.0(1,1)	7.0(1,1)	7.0(1,1)	7.0(1,1)	7.0(1,1)	7.0(1,1)
bsl	lr	7.0(1,1)	7.0(1,1)	7.0(1,1)	7.0(1,1)	7.0(1,1)	7.0(1,1)
bsl	lrl	9.0(1,1)	9.0(1,1)	9.0(4,1)	9.0(1,1)	9.0(1,1)	9.0(1,1)
eface	full	8.0(1,1)	8.0(1,1)	8.0(1,1)	8.0(1,1)	6.3(2,3)	8.0(1,1)
eface	lr	8.0(1,1)	8.0(1,1)	8.0(1,1)	8.0(1,1)	8.0(1,1)	8.0(1,1)
eface	lrl	7.9(1,1)	7.9(1,1)	5.4(8,1)	7.9(1,1)	6.1(3,4)	7.9(1,1)
twoh	full	4.9(1,1)	4.9(1,1)	4.9(1,1)	4.9(1,1)	1.8(4,4)	4.9(1,1)
twoh	lr	4.9(1,1)	4.9(1,1)	2.2(4,1)	4.9(1,1)	1.0(6,5)	4.9(1,1)
twoh	lrl	1.8(1,1)	1.8(1,1)	1.8(1,1)	1.8(1,1)	1.8(1,1)	1.8(1,1)
au20	full	55.6(1,2)	57.3(1,1)	46.2(6,1)	41.7(3,4)	39.1(6,3)	42.0(10,1)
au20	lr	57.3(1,1)	57.3(1,1)	46.2(7,1)	51.4(1,3)	42.2(4,3)	43.8(10,1)
au20	lrl	36.5(1,3)	49.8(1,1)	46.5(5,1)	36.5(1,3)	36.5(1,3)	36.5(1,3)
au40	full	50.0(3,5)	67.2(1,1)	56.5(6,1)	50.0(3,5)	44.0(8,5)	49.8(8,2)
au40	lr	67.2(1,1)	67.2(1,1)	55.9(9,1)	67.2(1,1)	55.0(4,3)	50.1(10,2)
au40	lrl	43.9(1,4)	57.5(1,1)	57.5(1,1)	43.9(1,4)	43.9(1,4)	46.8(1,3)
au60	full	70.5(1,1)	70.5(1,1)	58.5(8,1)	70.5(1,1)	49.8(7,4)	51.9(6,3)
au60	lr	70.5(1,1)	70.5(1,1)	59.0(9,1)	70.5(1,1)	53.8(4,5)	53.4(10,2)
au60	lrl	48.0(1,4)	61.2(1,1)	61.2(1,1)	48.0(1,4)	48.0(1,4)	48.0(1,4)
au80	full	74.8(1,1)	74.8(1,1)	61.7(9,1)	74.8(1,1)	56.1(7,4)	57.9(6,3)
au80	lr	74.8(1,1)	74.8(1,1)	64.8(9,1)	74.8(1,1)	52.6(9,5)	58.0(6,4)
au80	lrl	54.3(1,4)	66.4(1,1)	66.4(1,1)	53.1(1,5)	52.5(4,5)	54.3(1,4)

Holm düzeltmesi kullanılarak bulunan sonuçları Tablo 11 ve 12'de görebiliriz. Algoritmaları seçerken daha fazla tutucu olduğumuz için, arttırımlı algoritmaların tercih edildiğini görürüz. Bu sonuçlarda, Holm düzeltmesi kullanarak, iki algoritmaların aynı beklenen hatayı vermesini kabul etmesi zor olan bir istatistiksel test yapıyoruz, bu nedenle kolay $HMM_{1,1}$ durumundan başlayan arttırımlı algoritmalar derecelerine ve durum sayısına göre en iyidir. Avustralya veri kümesi dışındaki diğer veri kümelerinde, $HMM_{1,1}$ başarılı modeller verir, böylelikle çoğunlukla $HMM_{1,1}$ ve diğer yapılar arasında istatistiksel bir fark çıkmaz. Aynı kural düzeltme

olmayan duruma da uygulanır, ama bu durumda α üzerinde hiç düzeltme uygulamadığımız için, algoritma istatistiksel fark ve farklı yapılar bulabilir (bu veri kümelerinde sadece 2 au20-full ve au20-lr örneği olmasına rağmen). Holm düzeltmesi kullandığımızda, testler tutucu olmaya başlar ve hiç bir istatistiksel fark bulmaz. Bu durumda, en iyi MSUMO varyantı 1-Fw dir ve ziyaret edilen en az durum sayısı kullanarak buna ulaşır. Bw, dereceye göre en kötü algoritma olmasına rağmen, onun sırasının hala en iyi olduğunu görürüz, yani yine optimuma en yakın algoritmayı bulur. Düzeltme olmadığında veya Holm düzeltmesi olduğunda, bulunan yapıların ve hata oranlarının neredeyse aynı görürüz. Fark, dereceleri ve sıraları değiştiren optimum algoritmadır ve Tablo 9 ve 11’de görülebilir.

4. TARTIŞMA VE SONUÇLAR

4.1. Karar Ağaçlarında VC-Boyut Hesaplama

Projenin bu bölümünde tek değişkenli karar ağaçlarının VC-boyutunu iki algoritma kullanarak kestirdik. SRMexhaust diye adlandırdığımız ilk yaklaşımda, tek değişkenli karar ağaçlarının VC-boyutunu bulmak için kapsamlı arama algoritması kullandık. Algoritmayı üssel zaman karmaşıklığı yüzünden, en fazla 5 özellikli girdilere ve derinliği en fazla 5 olan karar ağaçlarına uygulayabildik. Bu sonuçlar daha sonra bir ağacın VC-boyutunu onun sol ve sağ alt ağaçlarının VC boyutuna, ağaçtaki düğüm sayısına ve veri kümesindeki özellik sayısına bağlayan doğrusal özyinelemeli uydurma bulmak için kullanıldı. SRMexper diye adlandırdığımız ikinci yaklaşımda, tek değişkenli karar ağaçlarının VC-boyutunu ortak ve optimize edilmiş deney dizaynı kullanarak kestiriyoruz. M düğümlü bir karar ağacının VC-boyutunu kestirmek için, karar ağacında M tane düğüm elde edene kadar hep en küçük logbenzerlikli düğümü genişlettiğimiz bir düğüm sınırlı ağaç algoritması öneriyoruz. Tek değişkenli karar ağaçlarının VC-boyutu düğüm sayısı ve özellik sayısının logaritmasıyla orantılıdır.

Bu kestirilen değerler yapısal risk azaltma tekniğiyle budamada kullanıldı ve önerdiğimiz budama tekniğini çapraz geçerlemeli budamayla karşılaştırdığımızda görüyoruz ki bizim kestirdiğimiz VC-boyutlarını kullanarak yapılan SRM budaması çok iyi çalışıyor ve çapraz geçerlemenin ek yükü veya eğitim kümesinden örnek çıkarma dışında CV budaması kadar iyi sonuçlar veren karar ağaçları buluyor.

İki yaklaşımı karşılaştırdığımızda, SRMexhaust'un yapıya bağlı olduğunu söyleyebiliriz, yani tek değişkenli karar ağacının VC-boyutunu sol ve sağ alt ağaçların VC-boyutunu kullanarak hesaplar. Diğer taraftan, SRMexper yapıdan bağımsızdır; aynı düğüm sayılı farklı karar ağaçları için aynı VC-boyutunu döndürür. Budamadaki performansları genellikle aynı olmasına rağmen, SRMexhaust SRMexper'den daha iyi geneller. Örneğin, SRMexper büyük ağaçları daha radikal bir biçimde budar ve bu bazen SRMexper'in performansını azaltır. İsminden de anlaşıldığı üzere, SRMexhaust tek değişkenli karar ağacının VC-boyutunu kestirmede kullandığımız tüm veri noktalarını (karar ağacı yapısı ve VC-boyutu) elde etmek için kapsamlı arama yapar ama SRMexper deney dizaynını kullanarak veri noktalarını elde eder (karar ağacındaki düğüm sayısı ve VC-boyutu). Bu nedenle, veri oluşturmada SRMexper SRMexhaust'dan daha hızlıdır.

Yaklaşımımızın ikili özellikten daha fazla özellik değerine sahip olan ağaçlara da genişletilebileceğini de bu projede gösterdik. Yaklaşımımız kolayca tek değişkenli ve sürekli özellikli karar ağaçlarına da genişletilebilir çünkü sürekli özellikler ayrık özelliklere çevrilebilir ve sonra ayrık özellikler için olan prosedür çalıştırılabilir.

Bu çalışmada, iki sınıf sınıflandırma durumları üzerinde duruldu. K-sınıf duruma genişletme VC-boyutun çoklu sınıflar için kestirilmesini gerektirir (RIDEŁLA VE ZUNINO, 2001). RIDEŁLA VE ZUNINO, 2001, VAPNIK VE DİĞERLERİ, 1994'in teorik altyapısını çoklu sınıflı problemlerin genelleştirilmiş hatasını bulmak üzere genişletti.

4.2. Saklı Markov Modellerinde Model Seçimi

Projemizin ikinci bölümünde SMM'lerde model seçimi için MSUMO algoritmasını önerdik. Algoritma ileri / geri aramayı tek ve çoklu işlemlerle gerçekleyen 5 varyanta sahiptir. Deneylerimizde gördük ki MSUMO tüm algoritmaları karşılaştırmadan ve tüm yapıları kapsamlı ayıklamadan optimum yapıya yakın yapılar bulur. MSUMO genel bir sistemdir ve diğer istatistiksel testlerle, düzeltmelerle ve arama stratejileriyle kullanılabilir. Durum uzayını tarar ve kabul edilebilir bir zamanda iyi bir model bulmaya çalışır.

Farklı SMM yapılarını beklenen hatalarını, karmaşıklıklarını ve optimum modele olan derece ve sıra ile mesafesini kullanarak karşılaştırdık. İstatistiksel test

kullanmanın etkilerini, düzeltme etkisini ve farklı arama stratejilerini, farklı SMM yapılarını ve dört veri kümesini beş MSUMO varyantı kullanarak inceledik.

Görüyoruz ki seçim kriterinin tutuculuğu artarsa (istatistiksel test kullanmadan, istatistiksel düzeltme kullanmadan, Holm düzeltmesi kullanarak), algoritma daha kolay modeller bulmaya eğilimli olur ve arama uzayında diğer durumlara geçme daha zor hale gelir. Bunların hepsi dizayn seçimleridir ve MSUMO varyantı seçimi elimizdeki probleme bağlıdır. Eğer daha kolay modellere ihtiyaç varsa, istatistiksel testli ve düzeltmeli MSUMO kullanmalıyız. Görüyoruz ki, bu veri kümelerinde, Bw algoritması en iyi yapıları bulur, ama diğer yapılara kıyasla daha çok sayıda durumları ziyaret eder.

Bir sonraki çalışma olarak, diğer düzeltmeleri kullanmayı ve son yapıdaki etkisini görmek için α 'yı değiştirmeyi planlıyoruz. Bu çalışmada, atlamalarımız birer adımdır, sonraki çalışmalarda çok adımlı atlamalar denenebilir. Bu durumda, her bir durumda atlanacak durum sayısına da karar vermek gerekir.

BİBLİYOGRAFYA

1. ARAN, O., Yıldız, O. T., Alpaydın, E., An incremental framework based on cross-validation for estimating the architecture of a multilayer perceptron, *International Journal of Pattern Recognition and Artificial Intelligence*, 3, 2, 1-38, (2009).
2. ARAN, O., Akarun, L., "Recognizing two handed gestures with generative, discriminative and ensemble methods via Fisher kernels," Multimedia Content Representation, Classification and Security International Workshop, MRCS 2006, Istanbul, Turkey, vol. LNCS 4015, (2006), pp. 159–166.
3. ARAN, O., Ari, I., Benoit, A., Campr, P., Carrillo, A. H., Fanard, F. X., Akarun, L., Caplier, A., Sankur, B. , Sigtutor: An interactive system for sign language tutoring, *IEEE Multimedia*, 16, 1, (2009).
4. ASUNCION, A., Newman, D. J., UCI machine learning repository, <http://www.ics.uci.edu/~mllearn/MLRepository.html>, (2007).
5. BIEM, A., A model selection criterion for classification: application to hmm topology optimization, Seventh International Conference on Document Analysis and Recognition, (2003), pp. 104-108.
6. BIEM, A., Ha, J. Y., Subrahmonia, J., A bayesian model selection criterion for hmm topology optimization, IEEE International Conference on Acoustics, Speech, and Signal Processing, vol. 1, (2002), pp. 989-992.
7. BOWDEN, R., British sign language dataset, <http://personal.ee.surrey.ac.uk/Personal/R.Bowden/sign/index.html>
8. CHERKASSKY, V., Mulier, F., *Learning From Data*, John Wiley and Sons, (1998).
9. CHERKASSKY, V., Ma, Y., Comparison of model selection for regression. *Neural Computation*, 15, 1691- 1714, (2003).
10. CHIEN, J. T., Furui, S., Predictive hidden markov model selection for speech recognition, *IEEE Transactions on Speech and Audio Processing*, 13, 3, 377–387, (2005).
11. GHAHRAMANI, Z., An introduction to hidden markov models and bayesian networks, *International Journal of Pattern Recognition and Artificial Intelligence*, 15, 1, 9-42, (2001).

12. HASTIE, T., Tibshirani, R., Friedman, J., *The elements of statistical learning*. Springer-Verlag, (2001).
13. HOLM, S., A simple sequentially rejective multiple test procedure, *Scandinavian Journal of Statistics*, 6, 65–70, (1979).
14. LI, W., Wu, C. F. J., Columnwise-pairwise algorithms with applications to the construction of supersaturated designs, *Technometrics*, 39, 171-179, (1997).
15. MARCEL, S., Just, A., IDIAP Two handed gesture dataset, <http://www.idiap.ch/~marcel/>.
16. MARTIN, J., Gibrat, J. F., Rodolphe, F., Choosing the optimal hidden markov model for secondary-structure prediction, *IEEE Intelligent Systems*, 20, 6, 19–25, (2005).
17. QUINLAN, J. R., Induction of decision trees, *Machine Learning*, 1, 81-106, (1986).
18. QUINLAN, J. R., *C4.5: Programs for Machine Learning*. Morgan Kaufmann, San Mateo, CA, (1993).
19. RABINER, L. R., A tutorial on hidden Markov models and selected applications in speech recognition, *IEEE Proceedings*, 77, 2, 257-285, (1989).
20. REYES-GOMEZ, M., Ellis, D., Selection, parameter estimation, and discriminative training of hidden markov models for general audio modeling, *International Conference on Multimedia and Expo*, vol. 1, (2003), pp. 73-76.
21. RIDELLA, S., Zunino, R., Empirical measure of multiclass generalization performance: The k-winner machine case, *IEEE Transactions on Neural Networks*, 12, 1525-1529, (2001).
22. SHAO, X., Cherkassky, V., Li, W., Measuring the vc-dimension using optimized experimental design, *Neural Computation*, 12, 1969-1986, (2000).
23. VAPNIK, V., *The Nature of Statistical Learning Theory*, Springer Verlag, New York, (1995).
24. VAPNIK, V., Levin, E., Cun, Y. L., Measuring the vc-dimension of a learning machine, *Neural Computation*, 6, 851-876, (1994).
25. YILDIZ, O. T., Alpaydın, E., Omnivariate decision trees, *IEEE Transactions on Neural Networks*, 12, 1539-1546, (2001).

26. YILDIZ, O. T., Alpaydın, E., Ordering and finding the best of $K > 2$ supervised learning algorithms, *IEEE Transactions on Pattern Analysis and Machine Intelligence.*, 28, 3, 392–402, (2006).